22CIPC504 · 21CIPC504

Automata Theory & Compiler Design

Previous Year Question Papers — Complete Solutions, organised unit-wise

Nov–Dec 2025
UVCE · UVCE-UG-2022
V Sem B.Tech CSE/ISE
Dec-25

April 2025
UVCE · UVCE-UG-2022
V Sem B.Tech CSE/ISE
Apr-25

July–Aug 2024
EN-024 · NEP Scheme
V Sem B.Tech CSE & AIML
Aug-24

Every question from all three papers is answered once, grouped by unit and topic. Questions that repeat across papers are merged, and the paper tags show where each one appeared. Answer length follows the marks: about 1 page for 8–10 marks and ½ page for 4–6 marks. Diagrams, transition tables and parsing tables are included wherever they help.

Contents

UNIT I — Finite Automata, Regular Expressions & Grammars

1.1 Define FA. Obtain a DFA to accept strings of a's and b's starting with ab(a+b)* with transition table. · 6 M Aug-24 Q1a

1.2 Draw a DFA to accept the set of all strings on Σ = {0,1} that either begin or end (or both) with the substring 01. · 10 M Dec-25 Q1a

1.3 Draw a DFA to accept the set of all strings of a's and b's ending with ab or ba. · 8 M Apr-25 Q1a

1.4 Design an NFA to recognize the set of strings abc, abd and aacd. · 6 M Dec-25 Q1b

1.5 Obtain an NFA to accept L = {w | w ∈ ababn or aban, n ≥ 0}. · 5 M Apr-25 Q2b

1.6 A toll booth charges a fee based on the sum of coins inserted and allows a car to pass only if the total is … · 10 M Dec-25 Q2a

1.7 Define CFG. Construct a DFA which accepts only those strings representing zero modulo five. · 8 M Aug-24 Q2a

1.8 Obtain a regular expression for L(R) = {w | w ∈ {0,1}*} with at least three consecutive 0's. · 4 M Dec-25 Q1c

1.9 Obtain a regular expression for strings of a's and b's whose third symbol from the right is 'a' and fourth symbol … · 4 M Apr-25 Q1c

1.10 Convert the given ε-NFA to an equivalent NFA (q0 with self-loop 0, q0 —ε→ q1, q1 with self-loop 1, q1 final). · 10 M Dec-25 Q2b Apr-25 Q2c

1.11 Convert the given NFA with ε-moves to DFA (q0: b→q0, ε→q1; q1: a→q1, ε→q2; q2: a→q1, b→q0; q2 final). · 7 M Aug-24 Q1c

1.12 Convert the given NFA to its equivalent DFA (q0 loop 0; q0 —0,1→ q1; q1 —0,1→ q2; q2 loop 1; q2 accepting). · 8 M Apr-25 Q1b

1.13 Convert the following NFA to DFA (P: 0→{P,Q}, 1→{P}; Q: 0,1→{R}; R: 0→{S}, 1→∅; S final: 0,1→{S}). · 7 M Aug-24 Q2b

1.14 Define grammar and explain the different types of grammar. · 7 M Aug-24 Q1b

1.15 Construct the leftmost and rightmost derivation for the string aaabbabbba using S → aB | bA, A → a | aS | bAA, B → b … · 5 M Apr-25 Q2a

1.16 Consider the grammar S → aS | aSbS | ε. Show that the derivation for the string aab is ambiguous. · 5 M Aug-24 Q2c

UNIT II — Introduction to Compilers & Lexical Analysis

2.1 Draw the structure of a compiler and describe the various phases of compilation. Show the output of each phase for … · 10 M Dec-25 Q3a Apr-25 Q3b Aug-24 Q4a

2.2 Analyse the given C program: identify the errors, the compiler phase that detects each one, why each occurs, and … · 6 M Dec-25 Q3b

2.3 Describe the grouping of compiler phases and their significance. · 5 M Apr-25 Q3c

2.4 Describe the role of a compiler. · 6 M Aug-24 Q3c

2.5 Who are the cousins of a compiler? Explain their roles. / Explain the different types of language processors. · 5–6 M Dec-25 Q4b Apr-25 Q3a

2.6 What is the difference between a compiler and an interpreter? · 4 M Dec-25 Q3c

2.7 List and explain the different compiler-construction tools and their purposes. · 6 M Dec-25 Q4a

2.8 Explain input buffering in detail. Describe the role of the forward pointer and the lexeme-begin pointer. How do … · 7–8 M Dec-25 Q4c Apr-25 Q4b Aug-24 Q4c

2.9 Explain the LEX tool with the structure of a Lex program. · 7 M Apr-25 Q4a Aug-24 Q3a

2.10 Explain the process of token recognition in lexical analysis. / Write a regular expression for relational operators … · 5 M Apr-25 Q4c Aug-24 Q5b

2.11 Describe the lexical errors and the various error-recovery strategies, with suitable examples. · 8 M Aug-24 Q6b

UNIT III — Syntax Analysis — Top-Down Parsing

3.1 What is a parser? Explain the role of a parser and its types in detail. · 7–8 M Dec-25 Q5a Apr-25 Q6c Aug-24 Q4a

3.2 Explain left-recursive grammars and left factoring with examples. · 6–7 M Dec-25 Q6a Aug-24 Q4b

3.3 (i) Convert S → a | ^ | (T), T → T,S | S into a non-left-recursive form. (ii) Left-factor S → iEtS | iEtSeS | a, E → b. · 6 M Apr-25 Q6b

3.4 Perform left factoring for the grammar A → abB | aB | cdg | cdeB | cdfB. · 7 M Aug-24 Q3b

3.5 Compute FIRST and FOLLOW for the grammar S → Bb | Cd, B → aB | ε, C → cC | ε. · 6 M Dec-25 Q6c Apr-25 Q5a

3.6 Construct the predictive parser (LL(1) table) for E → E+T | T, T → T*F | F, F → (E) | id (equivalently E → TE', E' → … · 8–10 M Dec-25 Q5b Apr-25 Q5b Aug-24 Q5a

3.7 Write a note on non-recursive descent parsing. / Explain the table-driven predictive parsing model. · 5–6 M Dec-25 Q5c Apr-25 Q6a

3.8 What is backtracking in parsing? Why is it inefficient? · 4 M Apr-25 Q5c

3.9 Discuss error-recovery strategies in predictive parsing. · 7 M Aug-24 Q6c

UNIT IV — Bottom-Up Parsing, SDD & Yacc

4.1 Explain bottom-up parsing in detail. Define augmented grammar and explain handle pruning. · 6 M Dec-25 Q8a Aug-24 Q8b

4.2 Trace the handle-pruning process while parsing the input "abbcdc" with a shift-reduce parser for S → aABc, A → Abc | … · 6 M Apr-25 Q7c

4.3 Perform shift-reduce parsing for the input string (a,(a,a)) using S → (L) | a, L → L,S | S. · 5 M Aug-24 Q5c

4.4 Differentiate between shift-reduce and operator-precedence parsers. · 5 M Aug-24 Q6a

4.5 Construct the LR(1) / CLR(1) parsing table for S → AA, A → aA | b. (Also: S → CC, C → aC | d.) · 7–10 M Dec-25 Q6b, Q8b Apr-25 Q8b Aug-24 Q8a

4.6 Construct the LALR(1) parsing table for S → AA, A → aA, A → b. · 8 M Dec-25 Q7a Apr-25 Q7a

4.7 Construct the LR(0) parsing table for E → E*B | E+B | B, B → 0 | 1. · 8 M Apr-25 Q8c

4.8 Differences between SLR, CLR and LALR parsers. · 4 M Aug-24 Q8c

4.9 Explain Syntax-Directed Definitions (SDD) with an example. Differentiate between synthesized and inherited … · 4–6 M Dec-25 Q7b Apr-25 Q8a Aug-24 Q7b

4.10 What are the applications of SDD? · 6 M Dec-25 Q8c

4.11 Write a note on Yacc. / Explain the working of the Yacc parser generator in detail. · 6–8 M Dec-25 Q7c Apr-25 Q7b Aug-24 Q7a

4.12 Discuss type checking with suitable examples. · 8 M Aug-24 Q7c

UNIT V — Intermediate Code, Run-Time Storage & Code Optimization

5.1 Explain the different types of intermediate code with an example. / Briefly explain the variants of syntax trees. · 7–8 M Aug-24 Q10a Apr-25 Q9a

5.2 Construct the syntax tree for the expression x * y − 5 + z. · 8 M Dec-25 Q9b

5.3 Construct the quadruples, triples and indirect triples of the expression a * −(b + c). · 8 M Dec-25 Q10b

5.4 Write the three-address code, quadruples, triples and indirect triples for: (1) a + b + (c + d) + d + e + a … · 10 M Apr-25 Q10b

5.5 (i) Translate A[i] := b and a := B[i] into triple representation. (ii) Generate the three-address code, compute the … · 8 M **Aug-24 Q9c**

5.6 Generate three-address code for: while (a < c and b > d) do if a = 1 then c = c + 1 else while a <= d do a … · 8 M **Apr-25 Q9b**

5.7 Construct a basic-block flow graph for: if (x > 10) y = x + 5; else y = x − 5; z = y * 2; and identify the number … · 7 M **Dec-25 Q9a**

5.8 Define a flow graph. Differentiate between a basic block and a flow graph. Explain the structure-preserving … · 4–5 M **Dec-25 Q10c** **Aug-24 Q9a**

5.9 What is a Directed Acyclic Graph (DAG)? Draw the DAG for t0 = a + b; t1 = t0 * c; d = t0 + t1. · 7 M **Aug-24 Q9b**

5.10 Construct the DAG for the block: a := b*c; d := b; e := d*c; b := e; f := b + c; g := f + d · 10 M **Apr-25 Q10a**

5.11 Construct the DAG for the basic block: t1 := 4*i; t2 := a[t1]; t3 := 4*i; t4 := b[t3]; t5 := t2*t4; t6 := prod + t5; … · 6 M **Aug-24 Q10b**

5.12 Explain the principal sources of optimization. · 5 M **Dec-25 Q9c**

5.13 Discuss copy propagation, dead-code elimination and strength reduction in detail. · 4–7 M **Apr-25 Q9c** **Aug-24 Q10c**

5.14 Explain storage organization in detail. · 8 M **Dec-25 Q10a**

UNIT I

Finite Automata, Regular Expressions & Grammars

DFA/NFA design · ε-NFA conversion · subset construction · regular expressions · derivations · ambiguity

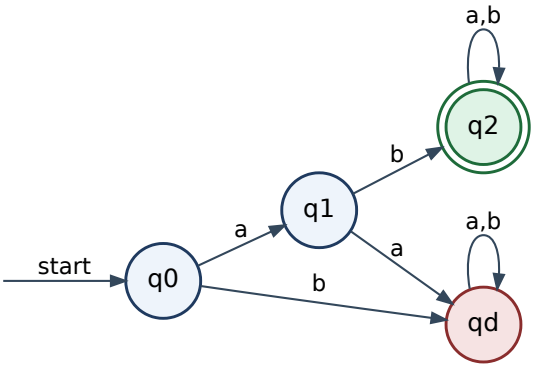
1.1 Define FA. Obtain a DFA to accept strings of a's and b's starting with ab(a+b)* with transition table. 6 M

Aug-24 Q1a

Finite Automaton (FA): An FA is a mathematical model of a machine with a finite number of states. It reads an input string one symbol at a time and accepts or rejects it. A DFA is a 5-tuple **M = (Q, Σ, δ, q0, F)**, where

- **Q** — finite set of states; **Σ** — input alphabet;
- **δ : Q × Σ → Q** — transition function; **q0 ∈ Q** — start state; **F ⊆ Q** — set of final states.

Language: L = ab(a+b)* = strings that begin with *ab*. After reading *a* then *b*, any further input is accepted. Any other prefix goes to a dead state.



δ	a	b
→q0	q1	qd
q1	qd	q2
*q2	q2	q2
qd	qd	qd

Check: "abba": q0→q1→q2→q2→q2 ✔ accepted. "ba": q0→qd→qd ✘ rejected.

1.2 Draw a DFA to accept the set of all strings on Σ = {0,1} that either begin or end (or both) with the substring 01. 10 M

Dec-25 Q1a

Idea: L = 01(0+1)* ∪ (0+1)*01. We build two "tracks" from one start state:

- **Begins-with-01 track:** q0 →0→ q1 →1→ **qF**. qF loops on 0,1 and stays final, because the string already begins with 01.
- **Ends-with-01 track:** once the string can no longer begin with 01, we only remember the last symbols: r0 = "no useful suffix", r1 = "last symbol is 0", **r2** = "last two symbols are 01" (final).

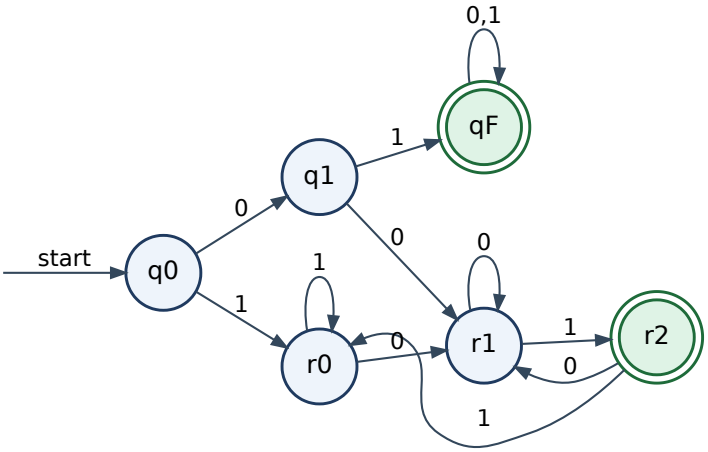


Fig. DFA for strings beginning or ending with 01 (final states: qF, r2)

State	0	1	Meaning
→q0	q1	r0	start
q1	r1	qF	read "0"
*qF	qF	qF	began with 01
r0	r1	r0	no suffix
r1	r1	r2	ends in 0
*r2	r1	r0	ends in 01

Verification

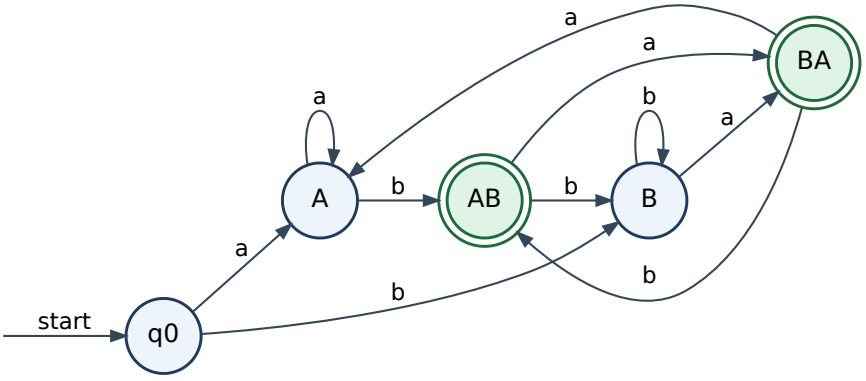
- "0110" → q1, qF, qF, qF ✔ (begins with 01)
- "1101" → r0, r0, r1, r2 ✔ (ends with 01)
- "0101" → q1, qF, qF, qF ✔ (both)
- "0010" → q1, r1, r2, r1 ✘
- "10" → r0, r1 ✘

1.3 Draw a DFA to accept the set of all strings of a's and b's ending with ab or ba.

8 M

Apr-25 Q1a

The DFA only needs to remember the **last two symbols**. States: **q0** (empty input), **A** (ends in a, not ba), **B** (ends in b, not ab), **AB** (ends with ab, final), **BA** (ends with ba, final).



δ	a	b
→q0	A	B
A	A	AB
B	BA	B
*AB	BA	B
*BA	A	AB

Traces

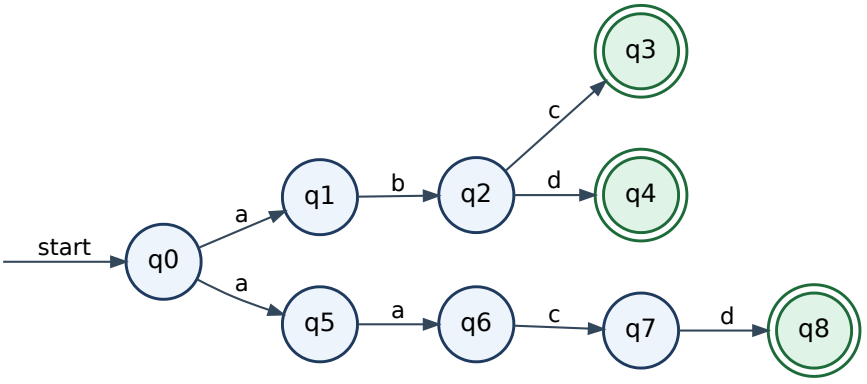
- "aab": A, A, AB ✔
- "abba": A, AB, B, BA ✔
- "aa": A, A ✘ "bbb": B,B,B ✘

Equivalent RE: (a+b)*(ab+ba)

1.4 Design an NFA to recognize the set of strings abc, abd and aacd. 6 M

Dec-25 Q1b

L = {abc, abd, aacd} is finite. The NFA starts one path per word, sharing the common prefix *ab*. The state q0 has **two moves on 'a'** (to q1 and q5), so the machine is non-deterministic.



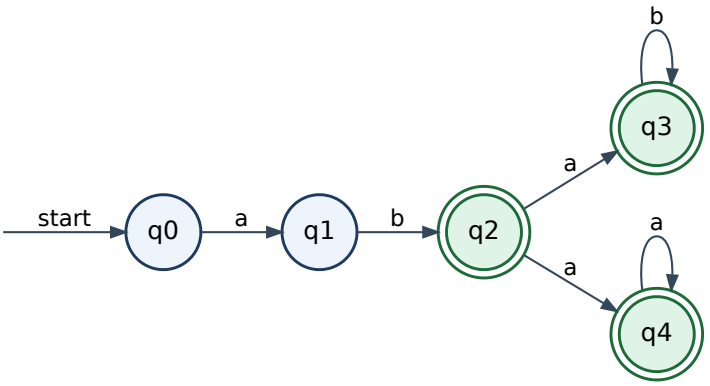
δ	a	b	c	d
→q0	{q1, q5}	∅	∅	∅
q1	∅	{q2}	∅	∅
q2	∅	∅	{q3}	{q4}
q5	{q6}	∅	∅	∅
q6	∅	∅	{q7}	∅
q7	∅	∅	∅	{q8}
*q3, *q4, *q8	∅	∅	∅	∅

M = ({q0…q8}, {a,b,c,d}, δ, q0, {q3, q4, q8}). For "aacd" the path q0→q5→q6→q7→q8 accepts. The branch through q1 dies on the second 'a'.

1.5 Obtain an NFA to accept L = {w | w ∈ abab^n or aba^n, n ≥ 0}. 5 M

Apr-25 Q2b

aba^n = {ab, aba, abaa, …} and abab^n = {aba, abab, ababb, …}. Both share the prefix **ab**. After it, the NFA guesses which branch to follow.



- **q2** final → accepts "ab" (aba^0).
- q2 →a→ **q3** (final, loop on b) → aba, abab, ababb … ($abab^n$)
- q2 →a→ **q4** (final, loop on a) → aba, abaa, … (aba^n, n ≥ 1)

Two separate branches are needed. One shared state looping on both a and b would wrongly accept strings like "abaab".

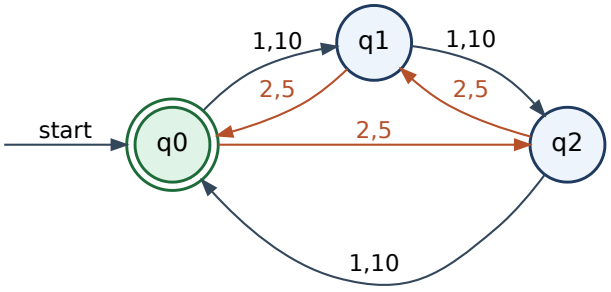
1.6 A toll booth charges a fee based on the sum of coins inserted and allows a car to pass only if the total is divisible by 3. Design a DFA to decide whether the sum is divisible by 3. — 10 M

Dec-25 Q2a

Assumption: the machine accepts Indian coins **Σ = {1, 2, 5, 10}** (rupees). We never need the actual total, only **total mod 3**. So the DFA needs 3 states:

- **q0** — sum ≡ 0 (mod 3) → start and **final** (gate opens)
- **q1** — sum ≡ 1 (mod 3) **q2** — sum ≡ 2 (mod 3)

Transition rule: $\delta(q_r, c) = q_{(r + c) \bmod 3}$. Since 1 ≡ 10 ≡ 1 and 2 ≡ 5 ≡ 2 (mod 3), coins {1,10} add 1 and coins {2,5} add 2.



δ	1	2	5	10
→*q0	q1	q2	q2	q1
q1	q2	q0	q0	q2
q2	q0	q1	q1	q0

M = ({q0,q1,q2}, {1,2,5,10}, δ, q0, {q0})

- Coins 5, 10: q0 →5→ q2 →10→ q0 ✔ total 15, gate opens
- Coins 2, 2: q0 → q2 → q1 ✘ total 4
- Coins 1, 2, 5, 1: q1, q0, q2, q0 ✔ total 9

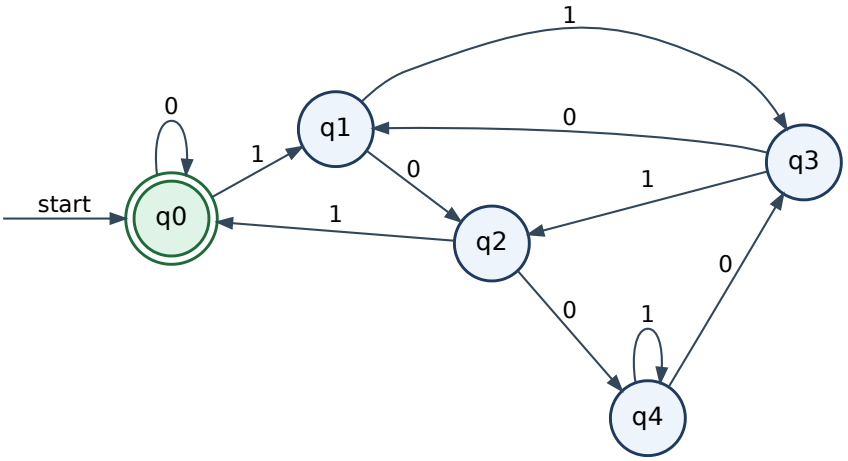
Note: The same 3-state design works for any set of coin values. A coin of value c simply moves the machine from residue r to (r + c) mod 3.

1.7 Define CFG. Construct a DFA which accepts only those strings representing zero modulo five. 8 M

Aug-24 Q2a

CFG: A context-free grammar is G = (V, T, P, S), where V = variables, T = terminals, P = productions of the form **A → α** (A ∈ V, α ∈ (V ∪ T)*), and S = start symbol. The left side always has a single variable, so a rule can be applied regardless of context. Example: S → aSb | ε generates $\{a^n b^n\}$.

DFA for binary numbers ≡ 0 (mod 5): If the value read so far is v with remainder r, reading bit d gives value 2v + d. So the new remainder is **(2r + d) mod 5**. The states q0…q4 stand for the remainders, and q0 is both the start state and the final state.



δ	0	1
→*q0	q0	q1
q1	q2	q3
q2	q4	q0
q3	q1	q2
q4	q3	q4

Check

- 1010 (=10): q1→q2→q0→q0 ✔
- 1111 (=15): q1→q3→q2→q0 ✔
- 111 (=7): q1→q3→q2 ✘ (rem 2)

1.8 Obtain a regular expression for L(R) = {w | w ∈ {0,1}*} with at least three consecutive 0's. 4 M

Dec-25 Q1c

The string must contain the substring **000** somewhere. Anything may come before it and anything after it.

R = (0 + 1)* 000 (0 + 1)*

Examples in L: 000, 10001, 110000. Not in L: 0010, 1001. Any extra 0's (e.g. 0000) are absorbed by (0+1)*.

1.9 Obtain a regular expression for strings of a's and b's whose third symbol from the right is 'a' and fourth symbol from the right is 'b'.

4 M

Apr-25 Q1c

Write the string's last four positions as: ... $\mathbf{x_4\ x_3\ x_2\ x_1}$. We need x_4 = b and x_3 = a, while x_2 and x_1 can be anything. The prefix before them is arbitrary.

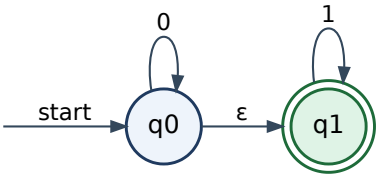
R = (a + b)* b a (a + b)(a + b)

Examples: "baab", "aababb", "babb". "abab" ✘ (its 4th symbol from the right is a).

1.10 Convert the given ε-NFA to an equivalent NFA (q0 with self-loop 0, q0 —ε→ q1, q1 with self-loop 1, q1 final).

10 M

Dec-25 Q2b Apr-25 Q2c



Given ε-NFA

Step 1 — ε-closures

ε-closure(q) = the set of states reachable from q using only ε-moves (q itself included).

- ε-closure(q0) = {q0, q1}
- ε-closure(q1) = {q1}

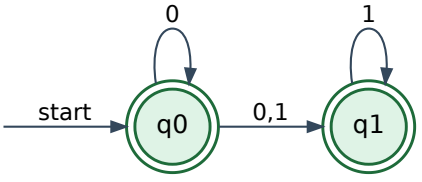
Step 2 — New transitions: δ'(q, a) = ε-closure(δ(ε-closure(q), a))

Computation	Result
δ'(q0,0) = ε-cl(δ({q0,q1},0)) = ε-cl({q0})	{q0, q1}
δ'(q0,1) = ε-cl(δ({q0,q1},1)) = ε-cl({q1})	{q1}
δ'(q1,0) = ε-cl(δ({q1},0)) = ε-cl(∅)	∅
δ'(q1,1) = ε-cl(δ({q1},1)) = ε-cl({q1})	{q1}

Step 3 — Final states

A state is final in the NFA if its ε-closure contains an original final state. ε-closure(q0) contains q1, so **q0 also becomes final**. F' = {q0, q1}.

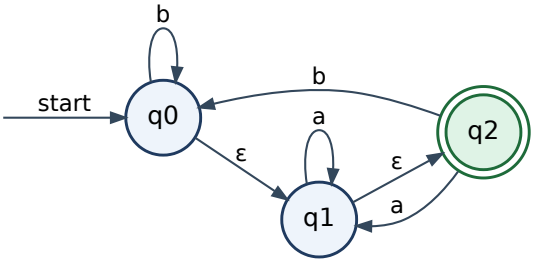
δ'	0	1
→*q0	{q0,q1}	{q1}
*q1	∅	{q1}



Result: the NFA has no ε-moves and accepts the same language, 0*1*. q0 is final because the empty string is accepted.

1.11 Convert the given NFA with ε-moves to DFA (q0: b→q0, ε→q1; q1: a→q1, ε→q2; q2: a→q1, b→q0; q2 final). 7 M

Aug-24 Q1c



Given ε-NFA

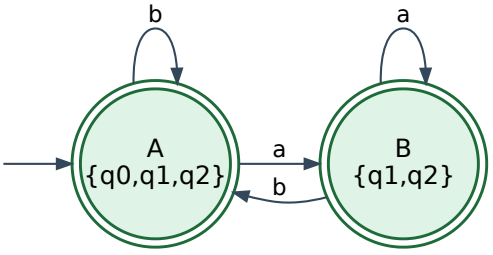
ε-closures: ε-cl(q0) = {q0, q1, q2}, ε-cl(q1) = {q1, q2}, ε-cl(q2) = {q2}

Subset construction (start state = ε-cl(q0) = **A = {q0,q1,q2}**):

DFA state	on a	on b
A = {q0,q1,q2}	δ = {q1} (from q1, q2) → ε-cl = {q1,q2} = **B**	δ = {q0} (from q0, q2) → ε-cl = {q0,q1,q2} = **A**
B = {q1,q2}	{q1} → ε-cl = {q1,q2} = **B**	{q0} (from q2) → ε-cl = **A**

Both A and B contain the final state q2, so **both are final**.

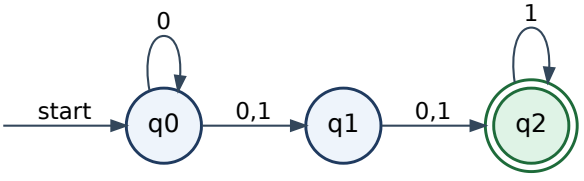
δ_D	a	b
→*A	B	A
*B	B	A



Observation: every DFA state is final and the DFA is complete, so it accepts (a+b)*. A and B are equivalent, so the minimal DFA has one state.

1.12 Convert the given NFA to its equivalent DFA (q0 loop 0; q0 —0,1→ q1; q1 —0,1→ q2; q2 loop 1; q2 accepting). 8 M

Apr-25 Q1b

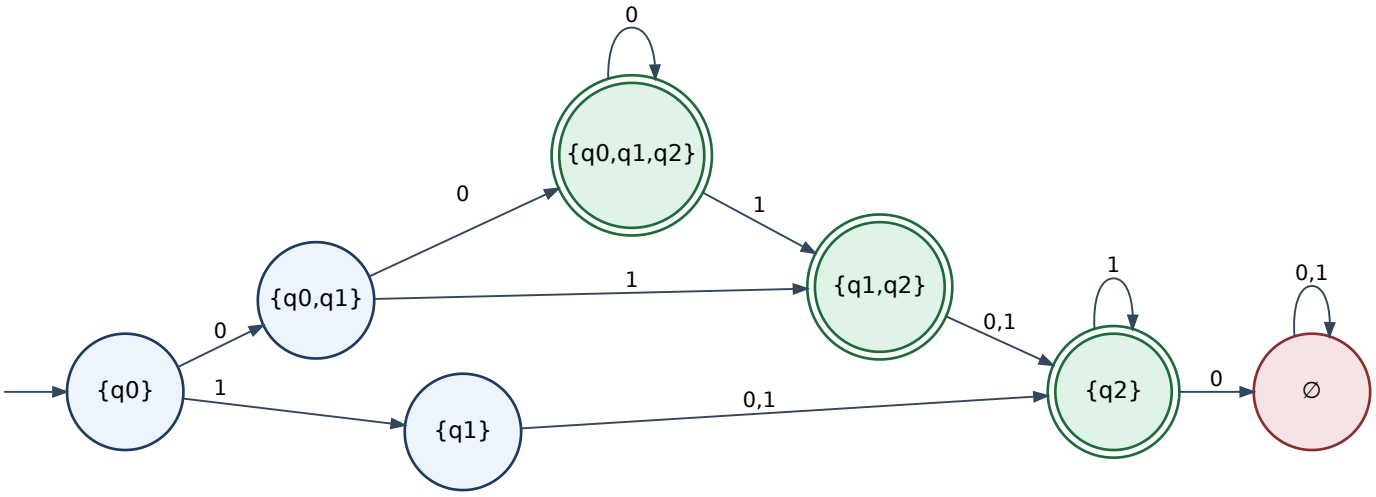


NFA transition table **Subset construction**

δ	0	1
→q0	{q0,q1}	{q1}
q1	{q2}	{q2}
*q2	∅	{q2}

DFA state	0	1
→[q0]	[q0,q1]	[q1]
[q0,q1]	[q0,q1,q2]	[q1,q2]
[q1]	[q2]	[q2]
*[q0,q1,q2]	[q0,q1,q2]	[q1,q2]
*[q1,q2]	[q2]	[q2]
*[q2]	∅	[q2]
∅ (dead)	∅	∅

Final DFA states = every subset that contains q2: [q0,q1,q2], [q1,q2], [q2].



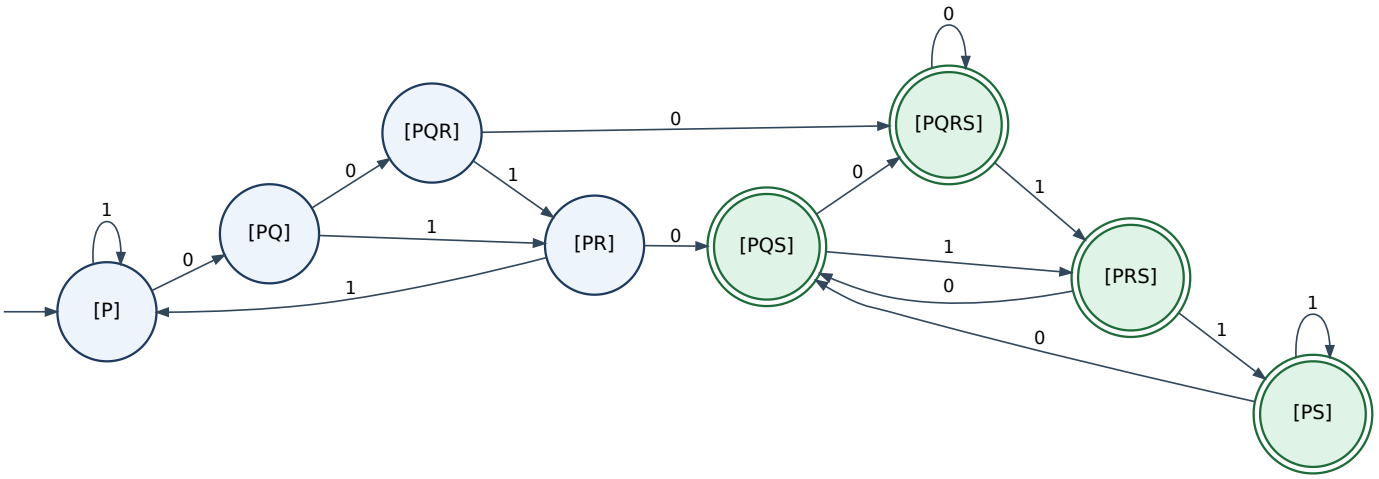
Equivalent DFA (7 states including the dead state ∅)

1.13 Convert the following NFA to DFA (P: 0→{P,Q}, 1→{P}; Q: 0,1→{R}; R: 0→{S}, 1→∅; S final: 0,1→{S}). 7 M

Aug-24 Q2b

Start with [P] and compute only the reachable subsets, one at a time (lazy subset construction):

DFA state	0	1	Working for 0
→[P]	[PQ]	[P]	δ(P,0) = {P,Q}
[PQ]	[PQR]	[PR]	{P,Q} ∪ {R}
[PQR]	[PQRS]	[PR]	{P,Q} ∪ {R} ∪ {S}
[PR]	[PQS]	[P]	{P,Q} ∪ {S}
*[PQRS]	[PQRS]	[PRS]	{P,Q} ∪ {R} ∪ {S} ∪ {S}
*[PQS]	[PQRS]	[PRS]	{P,Q} ∪ {R} ∪ {S}
*[PRS]	[PQS]	[PS]	{P,Q} ∪ {S} ∪ {S}
*[PS]	[PQS]	[PS]	{P,Q} ∪ {S}



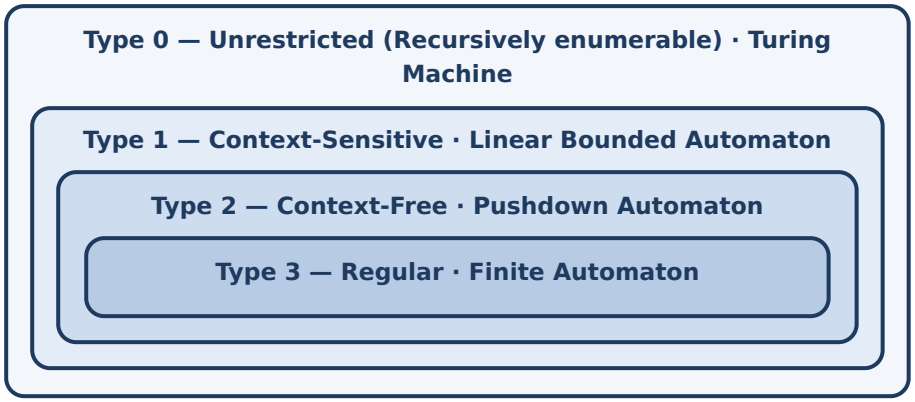
Resulting DFA — 8 states; final states are the ones containing S

1.14 Define grammar and explain the different types of grammar. 7 M

Aug-24 Q1b

Grammar: A grammar is a 4-tuple **G = (V, T, P, S)**. V is a finite set of variables (non-terminals), T is a finite set of terminals (V ∩ T = ∅), P is a finite set of productions α → β, and S ∈ V is the start symbol. The language of G is L(G) = { w ∈ T* | S ⇒* w }.

Chomsky Hierarchy — grammars are classified by the form of their productions:



Type	Production form	Machine	Example
Type 0 Unrestricted	α → β, α ∈ (V∪T)⁺ contains at least one variable; no restriction on β	Turing machine	AB → b
Type 1 Context-sensitive	αAβ → αγβ, |γ| ≥ 1 (non-contracting: |lhs| ≤ |rhs|)	Linear bounded automaton	aAb → aab; L = $a^n b^n c^n$
Type 2 Context-free	A → α, A ∈ V, α ∈ (V∪T)*	Pushdown automaton	S → aSb | ε
Type 3 Regular	A → aB | a (right-linear) or A → Ba | a (left-linear)	Finite automaton	S → aS | b

Each type is a proper subset of the one above it: Regular ⊂ CFL ⊂ CSL ⊂ REL. CFGs are used to specify programming-language syntax, and regular grammars or expressions are used to specify tokens.

1.15 Construct the leftmost and rightmost derivation for the string aaabbabbba using S → aB | bA, A → a | aS | bAA, B → b | bS | aBB.

5 M

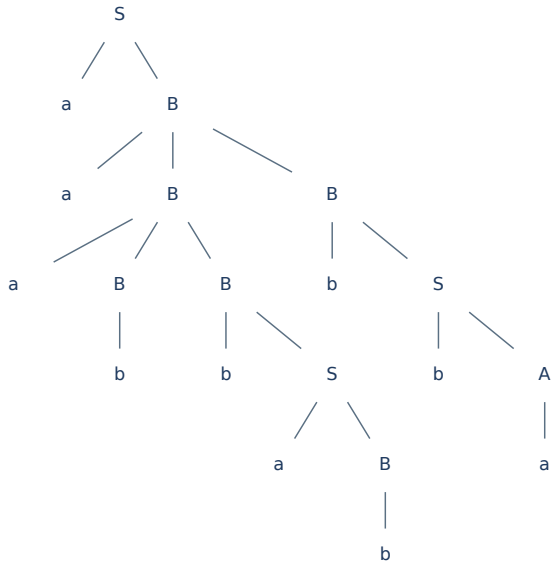
Apr-25 Q2a

Leftmost Derivation (LMD)

```
S ⇒ aB            (S→aB)
  ⇒ aaBB          (B→aBB)
  ⇒ aaaBBB        (B→aBB)
  ⇒ aaabBB        (B→b)
  ⇒ aaabbSB       (B→bS)
  ⇒ aaabbaBB      (S→aB)
  ⇒ aaabbabB      (B→b)
  ⇒ aaabbabbS     (B→bS)
  ⇒ aaabbabbbA    (S→bA)
  ⇒ aaabbabbba    (A→a)
```

Rightmost Derivation (RMD)

```
S ⇒ aB            (S→aB)
  ⇒ aaBB          (B→aBB)
  ⇒ aaBbS         (B→bS)
  ⇒ aaBbbA        (S→bA)
  ⇒ aaBbba        (A→a)
  ⇒ aaaBBbba      (B→aBB)
  ⇒ aaaBbSbba     (B→bS)
  ⇒ aaaBbaBbba    (S→aB)
  ⇒ aaaBbabbba    (B→b)
  ⇒ aaabbabbba    (B→b)
```



Parse tree (the same tree for both derivations; reading the leaves left to right gives a a a b b a b b b a)

1.16 Consider the grammar S → aS | aSbS | ε. Show that the derivation for the string aab is ambiguous.

5 M

Aug-24 Q2c

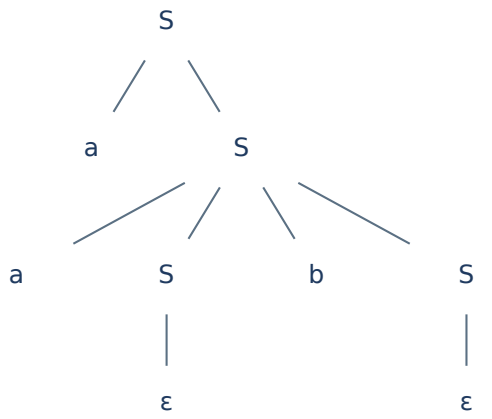
A grammar is **ambiguous** if some string has more than one leftmost derivation, or equivalently more than one parse tree. For w = aab:

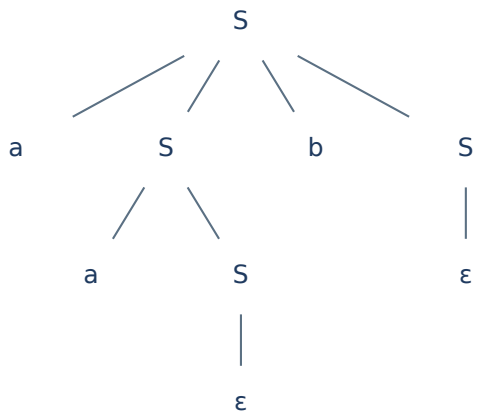
LMD 1

```
S ⇒ aS          (S→aS)
  ⇒ aaSbS       (S→aSbS)
  ⇒ aabS        (S→ε)
  ⇒ aab         (S→ε)
```

LMD 2

```
S ⇒ aSbS        (S→aSbS)
  ⇒ aaSbS       (S→aS)
  ⇒ aabS        (S→ε)
  ⇒ aab         (S→ε)
```





There are two different parse trees, and two different leftmost derivations, for the same string "aab". Hence the grammar **S → aS | aSbS | ε is ambiguous**. This is the "dangling-else" situation: 'b' can pair with either 'a'.

UNIT II

Introduction to Compilers & Lexical Analysis

Phases · language processors · compiler tools · input buffering · LEX · token recognition · lexical errors

2.1 Draw the structure of a compiler and describe the various phases of compilation. Show the output of each phase for the statement a = b + c * 60. 10 M

Dec-25 Q3a Apr-25 Q3b Aug-24 Q4a

A compiler translates a program written in a source language into an equivalent program in a target language. The work is done in two parts. **Analysis** (front end) breaks the source into pieces and builds an intermediate representation. **Synthesis** (back end) builds the target program from that representation. Each part is made of phases, and all phases share the **symbol table** and the **error handler**.

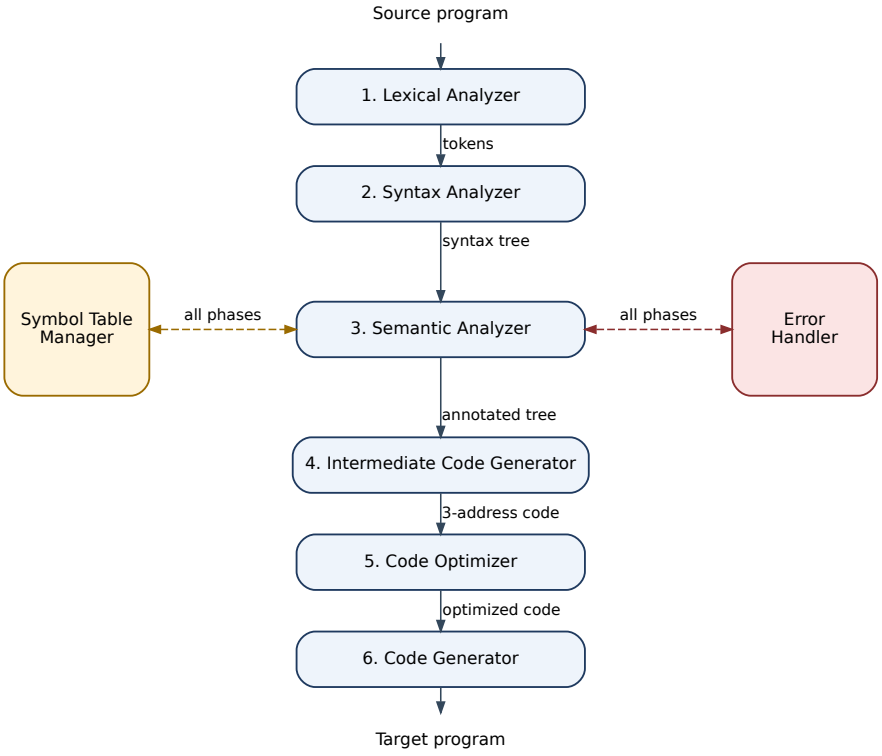


Fig. Phases of a compiler

1. **Lexical Analysis (Scanning):** reads the character stream and groups it into *lexemes*. For each lexeme it emits a token <token-name, attribute>. It also removes white space and comments and enters identifiers in the symbol table.
2. **Syntax Analysis (Parsing):** uses the tokens to build a *syntax tree* according to the grammar (CFG), and reports syntax errors.
3. **Semantic Analysis:** checks the tree for semantic consistency, mainly *type checking*. It inserts type conversions (coercions) where needed and records type information.
4. **Intermediate Code Generation:** produces a machine-independent, easy-to-translate IR, such as three-address code.

5. **Code Optimization:** improves the IR so the program runs faster or uses less memory (constant folding, removing redundant temporaries, etc.).
6. **Code Generation:** maps the IR to target instructions, allocating registers and memory.

Symbol table: stores each identifier with its attributes (type, scope, storage). **Error handler:** lets every phase report errors and recover.

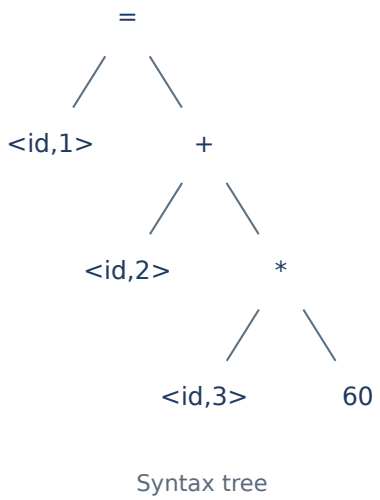
Translation of a = b + c * 60 (a, b, c are float)

Phase	Output
Lexical analyzer	`<id,1> <=> <id,2> <+> <id,3> <*> <number,60>` `Symbol table: 1 → a, 2 → b, 3 → c`
Syntax analyzer	Syntax tree (left figure below). * binds tighter than +.
Semantic analyzer	The integer 60 is converted to float with *inttofloat* (right figure).

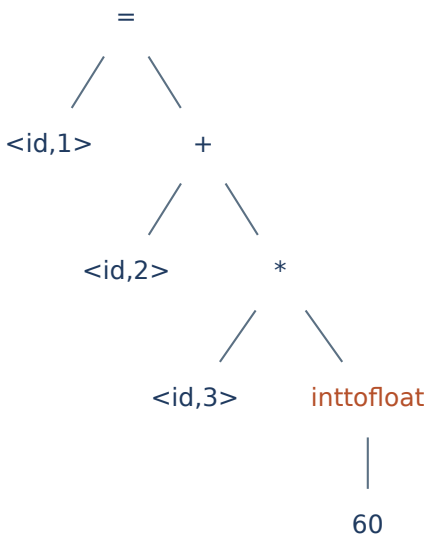
Intermediate code generator	`t1 = inttofloat(60)`
	`t2 = id3 * t1`
	`t3 = id2 + t2`
	`id1 = t3`

Code optimizer	`t1 = id3 * 60.0` `id1 = id2 + t1`

Code generator	`LDF R2, id3`
	`MULF R2, R2, #60.0`
	`LDF R1, id2`
	`ADDF R1, R1, R2`
	`STF id1, R1`



Syntax tree



After semantic analysis

For the Dec-25 variant a = b + c − 60, the steps are the same: tokens <id,1> <=> <id,2> <+> <id,3> <−> <num,60>; TAC t1 = inttofloat(60), t2 = id2 + id3, t3 = t2 − t1, id1 = t3; optimized code t1 = id2 + id3, id1 = t1 − 60.0.

2.2 Analyse the given C program: identify the errors, the compiler phase that detects each one, why each occurs, and write the corrected code. 6 M

Dec-25 Q3b

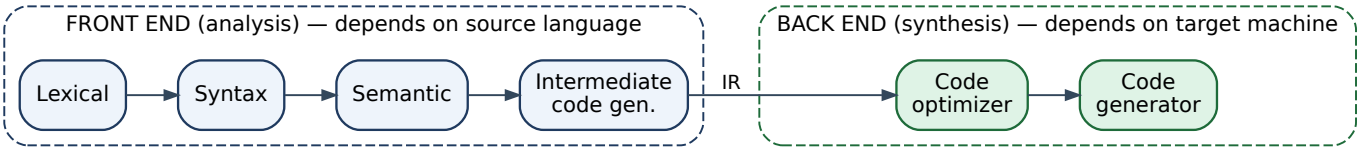
```
#include <stdio.h>
int main () {
    int 123 num = 10;
    int x = 5
    float y = "hello";
    print ("value of x: %d", x);
    return 0;
}
```

```
/* Corrected version */
#include <stdio.h>
int main() {
    int num123 = 10;
    int x = 5;
    char *y = "hello";   /* or: float y =
5.0; */
    printf("value of x: %d", x);
    return 0;
}
```

#	Error	Phase that detects it	Why it occurs
1	`int 123 num` — invalid identifier	Lexical (if written as one word, `123num` is a malformed token) / Syntax (with the space, the parser sees `int` followed by the number 123)	An identifier must start with a letter or _. A declaration expects an identifier after the type, not a constant.
2	`int x = 5` — missing semicolon	Syntax analysis	The grammar requires a declaration to end with ';'. The parser sees `float` where it expects ';'.
3	`float y = "hello";` — type mismatch	Semantic analysis (type checking)	A string literal (char*) cannot be assigned to a float. The types are incompatible.
4	`print(...)` — undefined function	Semantic analysis (implicit-declaration warning) and then the **linker** ("undefined reference to print")	The standard library function is `printf`. No function named `print` is declared or defined.

2.3 Describe the grouping of compiler phases and their significance. 5 M

Apr-25 Q3c



- **Front end (analysis):** lexical, syntax and semantic analysis plus intermediate code generation. It depends on the *source language* and is independent of the target machine.
- **Back end (synthesis):** code optimization and code generation. It depends on the *target machine* and is independent of the source language.
- **Passes:** several phases can be grouped into one pass that reads one input file and writes one output file. For example, lexing, parsing, semantic analysis and IR generation can form a single front-end pass. Fewer passes make compilation faster; more passes use less memory and allow more optimization.

Significance: (1) **Retargeting** — one front end + several back ends = one language on many machines. (2) Several front ends + one back end = many languages on one machine (e.g. GCC, LLVM). For *m* languages and *n* machines we need only m + n components instead of m × n compilers. (3) Each phase can be designed, tested and optimized separately.

2.4 Describe the role of a compiler. — 6 M

Aug-24 Q3c

A **compiler** is a program that reads a program in one language (the *source*) and translates it into an equivalent program in another language (the *target*, usually machine code). While doing so, it reports any errors in the source program.

- **Translation:** converts high-level constructs (loops, functions, objects) into low-level instructions the hardware can execute.
- **Error detection and reporting:** finds lexical, syntax and semantic errors and reports them with line numbers, so the programmer can fix them before the program runs.
- **Optimization:** produces efficient code, such as fewer instructions or better register use, without changing what the program means.
- **Abstraction:** lets programmers write portable, readable code without knowing machine details.
- **Symbol management:** keeps the symbol table (names, types, scopes, storage) used for type checking and memory allocation.
- **Support for the toolchain:** works with the preprocessor, assembler, linker and loader to produce an executable.

Source program → **Compiler** → Target program; then Input → **Target program** → Output. The target program runs independently, without the compiler.

2.5 Who are the cousins of a compiler? Explain their roles. / Explain the different types of language processors. — 5-6 M

Dec-25 Q4b Apr-25 Q3a

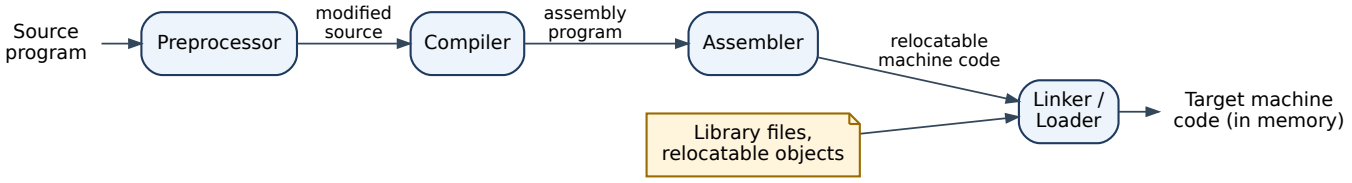


Fig. A language-processing system

Processor	Role
Preprocessor	Collects the source modules and expands macros (`#define`), file inclusion (`#include`), conditional compilation and language extensions. Its output is pure source code for the compiler.
Compiler	Translates the whole high-level program into assembly or machine code and reports errors.
Assembler	Translates assembly mnemonics into *relocatable machine code* (object files). It is usually two-pass: pass 1 builds the symbol table of labels, pass 2 generates the code.
Linker	Combines several object files and library routines. It resolves external references (for example a call to `printf`) and produces one executable.
Loader	Places the executable in memory, adjusts relocatable addresses (relocation), and starts execution.
Interpreter	Executes the source program statement by statement, without producing a separate target program (e.g. Python). Java uses a hybrid: it is compiled to bytecode, which the JVM interprets or JIT-compiles.

The preprocessor, assembler, linker and loader are called the **cousins of the compiler**: they work alongside it to turn source code into a running program.

2.6 What is the difference between a compiler and an interpreter? 4 M

Dec-25 Q3c

Compiler	Interpreter
Translates the entire program at once into target code	Translates and executes one statement at a time
Produces a separate object/executable file	Produces no target file
Reports errors after analysing the whole program	Stops at the first error it finds — easier to debug
Generated code runs **faster**	Execution is **slower** (it re-analyses code each time)
Uses more memory at compile time (builds object code)	Uses less memory; the source is needed at every run
Examples: C, C++, Go	Examples: Python, Ruby, JavaScript (classically)

2.7 List and explain the different compiler-construction tools and their purposes. 6 M

Dec-25 Q4a

1. **Scanner generators** (e.g. *Lex, Flex*): take regular-expression descriptions of tokens and produce a lexical analyzer (a DFA-driven C program).
2. **Parser generators** (e.g. *Yacc, Bison, ANTLR*): take a grammar description (CFG) and automatically produce a syntax analyzer, usually an LALR or LL parser.

3. **Syntax-directed translation engines:** produce routines that walk the parse tree and generate intermediate code, using translations attached to grammar productions.
4. **Code-generator generators:** take rules for translating each IR operation into target-machine instructions (template matching) and produce a code generator.
5. **Data-flow analysis engines:** collect information about how values flow through the program (live variables, reaching definitions). This is the key requirement for code optimization.
6. **Compiler-construction toolkits** (e.g. LLVM, GCC infrastructure): integrated sets of routines for building the different phases of a compiler.

2.8 Explain input buffering in detail. Describe the role of the forward pointer and the lexeme-begin pointer. How do they work together to recognize tokens? 7–8 M

Dec-25 Q4c Apr-25 Q4b Aug-24 Q4c

Why buffering? The lexer reads the source one character at a time, and often must *look ahead* one or more characters to decide the token. For example, after '>' it must check whether the next character is '=', and an identifier ends only when a non-letter is seen. Reading characters one by one from disk would be very slow. So the input is read in large blocks into a buffer.

Buffer pairs

Two buffers of N characters each (N = one disk block, e.g. 4096 bytes) are filled alternately with a single system read each. Two pointers are maintained:

- **lexemeBegin** — marks the start of the current lexeme being recognized.
- **forward** — scans ahead until a pattern match is found.



Fig. Buffer pair with sentinels (eof) at the end of each half

How the two pointers work together

1. At the start of each lexeme, both pointers point to its first character.
2. **forward** moves right one character at a time, as long as the characters can still form a valid token (following the DFA or transition diagram).
3. When a lexeme is complete (e.g. a blank or an operator follows an identifier), forward may have gone one character too far, so it is **retracted** by one.
4. The characters between lexemeBegin and forward form the **lexeme**. The token <name, attribute> is returned to the parser.
5. lexemeBegin is set to the character just after the lexeme, and scanning continues.

Sentinels

Without sentinels, every time forward advances we must make *two* tests: "is it the end of the buffer?" and "which character is it?". Placing a special **eof sentinel** at the end of each half means usually only one test is needed:

```
switch (*forward++) {
  case eof:
     if (forward is at end of first buffer)  { reload second buffer; forward = beginning of
second buffer; }
     else if (forward is at end of second buffer) { reload first buffer; forward = beginning
of first buffer; }
     else  /* eof within a buffer: real end of input */  terminate lexical analysis;
     break;
  /* cases for the other characters */
}
```

Limitation: a lexeme longer than N characters cannot be handled, and lookahead is limited to N characters.

2.9 Explain the LEX tool with the structure of a Lex program. 7 M

Apr-25 Q4a Aug-24 Q3a

Lex (or its newer version Flex) is a **scanner generator**. The programmer writes token patterns as regular expressions together with actions. Lex converts them into an NFA, then a DFA, and outputs C code, `lex.yy.c`, that simulates the DFA.

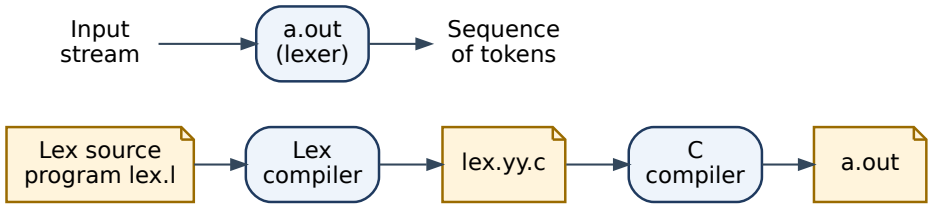


Fig. Creating a lexical analyzer with Lex

Structure of a Lex program

```
declarations
%%
translation rules
%%
auxiliary functions
```

- **Declarations:** C declarations in `%{ … %}` (header files, variables, token constants) and *regular definitions* such as `digit [0-9]`.
- **Translation rules:** each has the form `Pattern { Action }`. The action is C code that runs when the pattern matches.
- **Auxiliary functions:** user functions such as `main()`, `yywrap()`, `installID()`.

```
%{
#include <stdio.h>
%}
digit   [0-9]
letter  [a-zA-Z_]
%%
[ \t\n]+                    { /* skip white space */ }
if|else|while               { printf("KEYWORD: %s\n", yytext); }
{letter}({letter}|{digit})* { printf("ID: %s\n", yytext); }
{digit}+                    { printf("NUMBER: %s\n", yytext); }
"<="|">="|"=="|"<"|">"     { printf("RELOP: %s\n", yytext); }
.                           { printf("OTHER: %s\n", yytext); }
%%
int yywrap() { return 1; }
int main()   { yylex(); return 0; }
```

Conflict rules: (1) the *longest* matching prefix is preferred; (2) if two patterns match the same length, the one listed *first* wins. That is why keywords are listed before identifiers. Useful built-ins: `yytext` (the lexeme), `yyleng` (its length), `yylex()` (the scanner), `yylval` (the attribute value passed to the parser).

2.10 Explain the process of token recognition in lexical analysis. / Write a regular expression for relational operators and design a transition diagram for them. — 5 M

Apr-25 Q4c Aug-24 Q5b

Terms: a **token** is a pair <token-name, attribute> (e.g. <id, ptr>). A **pattern** describes the form of a token's lexemes (usually a regular expression). A **lexeme** is the actual character sequence that matches a pattern (e.g. "count").

Process: (1) token patterns are written as regular definitions; (2) each one is converted into a **transition diagram** (a DFA with actions); (3) the lexer simulates these diagrams on the input buffer, uses the longest match, and retracts when it has read one extra character (shown by *); (4) it returns the token and installs identifiers or numbers in the symbol table. Keywords are either pre-loaded into the symbol table or given their own diagrams.

Regular definitions:

```
relop  →  <  |  <=  |  =  |  <>  |  >  |  >=
id     →  letter ( letter | digit )*
number →  digit+ ( . digit+ )? ( E (+|-)? digit+ )?
```

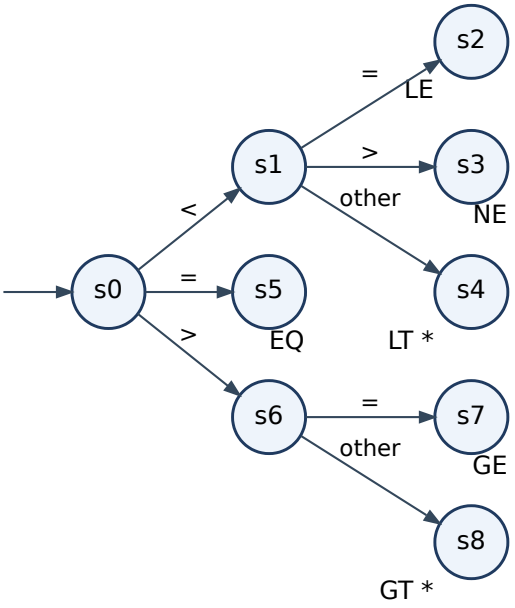


Fig. Transition diagram for relop (* = retract forward by one; return <relop, attribute>)

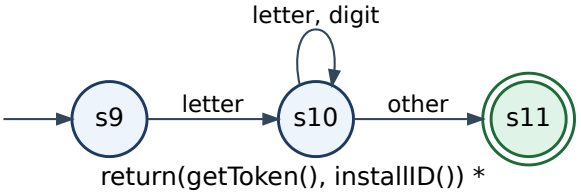


Fig. Transition diagram for identifiers and keywords

2.11 Describe the lexical errors and the various error-recovery strategies, with suitable examples. 8 M

Aug-24 Q6b

A **lexical error** occurs when no token pattern matches a prefix of the remaining input. The lexer sees the source only locally, so it can catch few errors. For example, in `fi (a == f(x))` it cannot tell that `fi` is a misspelled `if`: it is a valid identifier. Typical lexical errors:

- Illegal characters: `int a = 5 $ 3;` or `@count`
- Malformed identifiers or numbers: `2abc`, `3.14.5`, `1e`
- Unterminated strings or comments: `"hello`, `/*` … with no closing `*/`
- Identifiers or literals that are too long (exceed the compiler's limit)

Error-recovery strategies in the lexer

Strategy	Description	Example
Panic mode	Delete successive characters from the remaining input until a well-formed token is found. Simple and never loops forever.	`a = 5 $$ + 3` → skip `"$$"`
Delete one character	Remove an extraneous character	`whi#le` → `while`
Insert a missing character	Add a character to form a valid token	`"hello` → `"hello"`
Replace a character	Change one character into another	`1o0` → `100` (letter o replaced by digit 0)
Transpose two adjacent characters	Swap neighbours	`fi` → `if`, `esle` → `else`

The **minimum-distance** idea: choose the smallest number of such single-character transformations that turns the input into a valid lexeme. This is expensive, so in practice panic mode is used most.

General (parser-level) strategies: panic mode (skip to a synchronizing token such as ';' or '}'), phrase-level recovery (local corrections such as inserting a missing ';'), error productions (grammar rules for common mistakes), and global correction (the least-cost change to the whole program).

UNIT III

Syntax Analysis — Top-Down Parsing

Role of the parser · left recursion & factoring · FIRST/FOLLOW · LL(1) tables · backtracking · error recovery

3.1 What is a parser? Explain the role of a parser and its types in detail. 7–8 M

Dec-25 Q5a Apr-25 Q6c Aug-24 Q4a

A **parser** (syntax analyzer) is the second phase of a compiler. It takes the token stream from the lexical analyzer and checks whether the tokens can be generated by the grammar of the source language. If they can, it builds a **parse tree** (or syntax tree) for the rest of the front end.

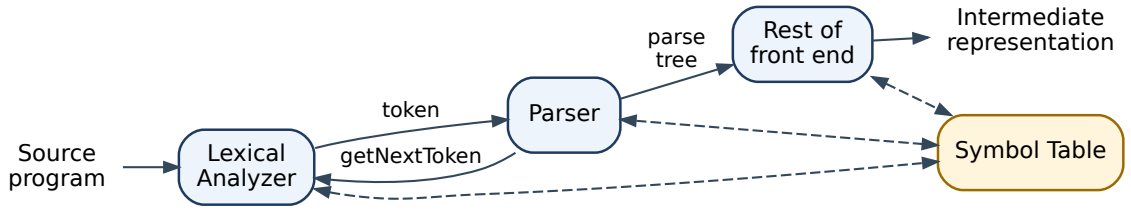


Fig. Position of the parser in the compiler model

Role of the parser

- Gets tokens on demand from the lexer (`getNextToken`) and verifies the syntax using the CFG.
- Builds the parse tree or syntax tree and passes it to semantic analysis and IR generation.
- Reports syntax errors clearly (line and position) and **recovers** from common errors so it can continue and find more of them.
- Collects information into the symbol table and may perform type checking or translation (syntax-directed translation).

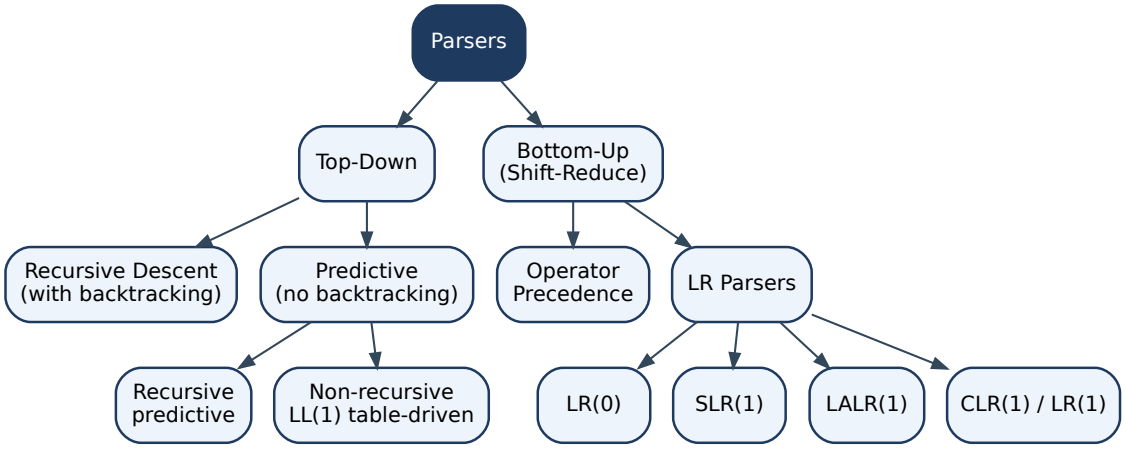


Fig. Classification of parsers

1. Top-down parsers build the parse tree from the root (start symbol) down to the leaves. They follow a *leftmost derivation*.

- **Recursive descent with backtracking:** one procedure per non-terminal. It tries alternatives one by one and backtracks on failure. Simple but slow.

- **Predictive parsing (no backtracking):** the right production is chosen by looking at one lookahead token. It needs a grammar with no left recursion that has been left-factored, i.e. an LL(1) grammar. It can be recursive-predictive or **non-recursive (table-driven) LL(1)**, which uses an explicit stack and a parsing table.

2. Bottom-up parsers build the tree from the leaves up to the root. They trace a *rightmost derivation in reverse*, using shift and reduce actions.

- **Operator-precedence parser:** for operator grammars (no ε-productions and no two adjacent non-terminals). It uses the precedence relations $<\cdot$, $\doteq$, $\cdot>$.
- **LR parsers:** LR(0), SLR(1), LALR(1) and CLR(1). They are the most powerful practical parsers and are used by Yacc and Bison.

3.2 Explain left-recursive grammars and left factoring with examples. 6–7 M

Dec-25 Q6a Aug-24 Q4b

Left recursion

A grammar is **left recursive** if it has a non-terminal A such that $A \Rightarrow^{+} A\alpha$. With *immediate* left recursion, $A \to A\alpha$. A top-down parser expanding A would call A again without consuming any input, so it loops forever. Left recursion must therefore be removed.

Rule: $A \to A\alpha_1 \mid A\alpha_2 \mid \ldots \mid \beta_1 \mid \beta_2 \mid \ldots \implies$ **$A \to \beta_1 A' \mid \beta_2 A' \mid \ldots$** and **$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \varepsilon$**

Example: $E \to E + T \mid T \implies E \to TE',\quad E' \to +TE' \mid \varepsilon$. Similarly $T \to T * F \mid F \implies T \to FT', T' \to *FT' \mid \varepsilon$.

Left factoring

When two alternatives of A start with the same prefix, a predictive parser cannot choose between them with one lookahead symbol. **Left factoring** takes out the common prefix and delays the decision until enough input has been seen.

Rule: $A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma \implies$ **$A \to \alpha A' \mid \gamma$**, **$A' \to \beta_1 \mid \beta_2$**

Example (dangling else): $S \to iEtS \mid iEtSeS \mid a \implies S \to iEtSS' \mid a,\quad S' \to eS \mid \varepsilon$.

Difference: left recursion causes *infinite looping* in top-down parsers; a missing left factoring causes *non-determinism* (backtracking). Both must be fixed to obtain an LL(1) grammar.

3.3 (i) Convert S → a | ^ | (T), T → T,S | S into a non-left-recursive form. (ii) Left-factor S → iEtS | iEtSeS | a, E → b. 6 M

Apr-25 Q6b

(i) S has no left recursion. T → T,S | S is immediately left recursive, with α = ",S" and β = "S". Applying the rule:

S → a | ^ | (T) T → S T' T' → , S T' | ε

(ii) The common prefix of the first two S-alternatives is α = iEtS, with $\beta_1 = \varepsilon$ and $\beta_2 = eS$:

S → iEtS S' | a S' → eS | ε E → b

3.4 Perform left factoring for the grammar A → abB | aB | cdg | cdeB | cdfB. 7 M

Aug-24 Q3b

Step 1: Group the alternatives by common prefix: {abB, aB} share **a**; {cdg, cdeB, cdfB} share **cd**.

Step 2: Factor out **a**: A → aA' | cdg | cdeB | cdfB, A' → bB | B

Step 3: Factor out **cd**: A → aA' | cdA'', A'' → g | eB | fB

A → aA' | cdA''
A' → bB | B
A'' → g | eB | fB

Now no two alternatives of any non-terminal share a common prefix, so a predictive parser can choose a production with one lookahead symbol.

3.5 Compute FIRST and FOLLOW for the grammar S → Bb | Cd, B → aB | ε, C → cC | ε. 6 M

Dec-25 Q6c Apr-25 Q5a

FIRST rules

- If X is a terminal, FIRST(X) = {X}.
- If X → ε, add ε to FIRST(X).
- If X → $Y_1Y_2\ldots$, add FIRST(Y_1) − {ε}. If $Y_1 \Rightarrow^* \varepsilon$, also add FIRST($Y_2$), and so on.

FOLLOW rules

- Put $ in FOLLOW(S).
- For A → αBβ, add FIRST(β) − {ε} to FOLLOW(B).
- For A → αB, or A → αBβ with $\beta \Rightarrow^* \varepsilon$, add FOLLOW(A) to FOLLOW(B).

FIRST:

- FIRST(B) = {a, ε} (B → aB gives a; B → ε gives ε)
- FIRST(C) = {c, ε}
- FIRST(S): from S → Bb: FIRST(B) − ε = {a}, and B ⇒ ε so add b. From S → Cd: {c}, and C ⇒ ε so add d. ⇒ FIRST(S) = {a, b, c, d}

FOLLOW:

- FOLLOW(S) = {$} (start symbol; S never appears on a right side)
- FOLLOW(B): in S → Bb, b follows B; in B → aB, add FOLLOW(B) itself. ⇒ {b}
- FOLLOW(C): in S → Cd ⇒ {d}

Non-terminal	FIRST	FOLLOW
S	{a, b, c, d}	{$}
B	{a, ε}	{b}
C	{c, ε}	{d}

The predictive table built from these has no conflicts (M[S,a] = M[S,b] = S→Bb; M[S,c] = M[S,d] = S→Cd; M[B,a] = B→aB; M[B,b] = B→ε; M[C,c] = C→cC; M[C,d] = C→ε), so the grammar is LL(1).

3.6 Construct the predictive parser (LL(1) table) for E → E+T | T, T → T*F | F, F → (E) | id (equivalently E → TE', E' → +TE' | ε, T → FT', T' → *FT' | ε, F → id | (E)). Check whether it is LL(1) and show the moves of the parser on id + id * id. 8-10 M

Dec-25 Q5b Apr-25 Q5b Aug-24 Q5a

Step 1 — Remove left recursion

E → E+T | T and T → T*F | F are left recursive. A top-down parser would loop forever, so **the original grammar is not LL(1)**. Using A → Aα | β ⇒ A → βA', A' → αA' | ε:

```
E  → T E'          E' → + T E' | ε
T  → F T'          T' → * F T' | ε
F  → ( E ) | id
```

Step 2 — FIRST and FOLLOW

NT	FIRST	FOLLOW
E	{ (, id }	{), $ }
E'	{ +, ε }	{), $ }
T	{ (, id }	{ +,), $ }
T'	{ *, ε }	{ +,), $ }
F	{ (, id }	{ *, +,), $ }

Step 3 — Parsing table M[A, a]

For each A → α: add it to M[A, a] for every a ∈ FIRST(α). If ε ∈ FIRST(α), add it to M[A, b] for every b ∈ FOLLOW(A).

NT	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id			F→(E)		

LL(1) check: every cell has at most one production (no multiple entries), so the transformed grammar **is LL(1)**.

Step 4 — Moves on w = id + id * id $

Stack (top at right)	Input	Action / Output
$E	id+id*id$	E → TE'
$E'T	id+id*id$	T → FT'
$E'T'F	id+id*id$	F → id
$E'T'id	id+id*id$	match id
$E'T'	+id*id$	T' → ε
$E'	+id*id$	E' → +TE'
$E'T+	+id*id$	match +
$E'T	id*id$	T → FT'
$E'T'F	id*id$	F → id
$E'T'id	id*id$	match id
$E'T'	*id$	T' → *FT'
$E'T'F*	*id$	match *
$E'T'F	id$	F → id
$E'T'id	id$	match id
$E'T'	$	T' → ε
$E'	$	E' → ε
$	$	**ACCEPT**

3.7 Write a note on non-recursive descent parsing. / Explain the table-driven predictive parsing model.

5-6 M

Dec-25 Q5c Apr-25 Q6a

A **non-recursive predictive parser** (LL(1) parser) simulates a leftmost derivation. Instead of recursive procedure calls, it keeps an **explicit stack**. It is driven by a parsing table built from FIRST and FOLLOW.

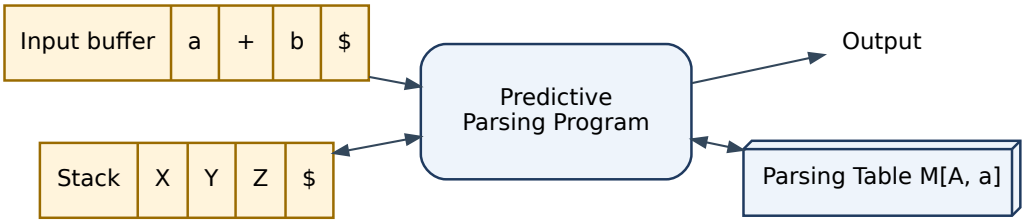


Fig. Model of a table-driven predictive parser

- **Input buffer:** the string to be parsed, followed by the end-marker $.
- **Stack:** grammar symbols, with $ at the bottom. At the start it holds $S (S on top).
- **Parsing table M[A, a]:** a 2-D array whose entries are productions or error.
- **Parsing program:** repeatedly looks at X (top of stack) and a (current input symbol).

```
push $ and then S onto the stack;  a = first input symbol;
while ( X = top of stack ) ≠ $ :
    if X == a            : pop X;  a = next input symbol        /* match */
    else if X is terminal: error()
    else if M[X, a] is error : error()
    else if M[X, a] = X → Y1 Y2 … Yk :
         output the production;  pop X;
         push Yk, …, Y2, Y1   (Y1 on top)
if a == $ : accept
```

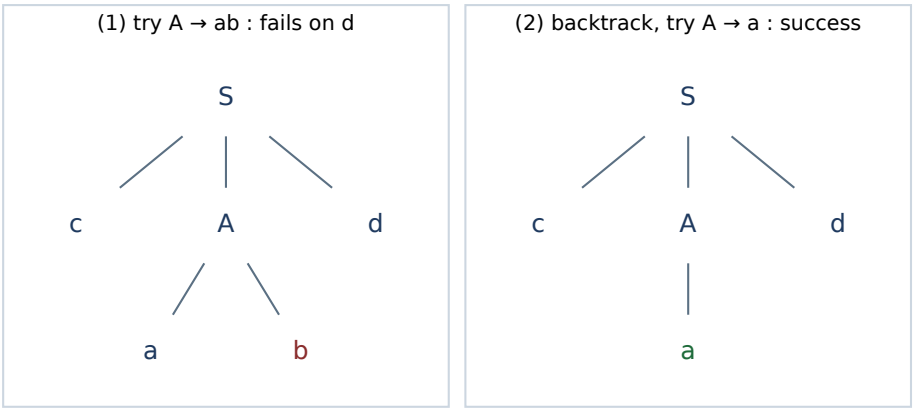
Advantages: no recursion overhead, no backtracking, linear time O(n), and easy to generate automatically. **Limitation:** works only for LL(1) grammars, so left recursion must be removed and the grammar left-factored. (See 3.6 for a complete trace.)

3.8 What is backtracking in parsing? Why is it inefficient? 4 M

Apr-25 Q5c

Backtracking happens in recursive-descent parsing when the parser picks one alternative of a non-terminal, finds it does not match the input, and then *undoes* its work: it resets the input pointer and tries the next alternative.

Example: S → cAd, A → ab | a, input **cad**.



Why it is inefficient:

- In the worst case the running time is **exponential**, because the same input is re-scanned many times for different alternatives.
- Any semantic actions already performed (symbol-table entries, generated code) must be undone.
- Error reporting is poor: it is hard to tell where the real error is, because every failure looks like "try the next alternative".
- It cannot handle left-recursive grammars (infinite loop).

Predictive (LL(1)) and LR parsers avoid backtracking by using lookahead tokens and a parsing table.

3.9 Discuss error-recovery strategies in predictive parsing. 7 M

Aug-24 Q6c

An LL(1) parser detects an error when (a) the terminal on top of the stack does not match the input symbol, or (b) the entry M[A, a] for non-terminal A and input a is empty.

1. Panic-mode recovery

Skip input symbols until a token from a chosen **synchronizing set** appears. Heuristics for building the set of A:

- Put all symbols of **FOLLOW(A)** into the sync set. On finding one of them, **pop A** and continue.
- Add symbols of FIRST(A). On seeing one of them, resume parsing A.
- Add keywords that start statements (if, while, ;) to the sync sets of lower-level constructs.
- If a terminal on top of the stack does not match, pop it ("insert" the missing token) and report it.
- If A ⇒ ε, the ε-production can be used as a default.

Example (expression table of 3.6): mark M[A, b] = **synch** for every b ∈ FOLLOW(A) where the cell is empty, e.g. M[E,)] = M[E, $] = synch, M[F, +] = M[F, *] = synch. On input `) id * + id`: the ')' is skipped (error); at '+' with F on the stack, the entry is synch, so F is popped and parsing continues.

2. Phrase-level recovery

Fill the blank entries of the table with pointers to **error routines**. These may insert, delete or change input symbols, or pop the stack, and they issue a suitable message (e.g. "missing operand", "unbalanced parenthesis"). Care is needed to avoid infinite loops: every action should consume input or shrink the stack.

UNIT IV

Bottom-Up Parsing, Syntax-Directed Translation & Yacc

Shift-reduce · handle pruning · LR(0) · CLR(1) · LALR(1) · SDD · type checking · Yacc

4.1 Explain bottom-up parsing in detail. Define augmented grammar and explain handle pruning. 6 M

Dec-25 Q8a Aug-24 Q8b

Bottom-up parsing builds the parse tree starting from the leaves (the input string) and works up to the root (the start symbol). At each step a substring that matches the right side of a production is **reduced** to the non-terminal on the left. Read in reverse, the sequence of reductions is a **rightmost derivation**.

Shift-reduce parsing is the general form. It uses a stack and an input buffer, both bottomed by $. There are four actions:

- **Shift** — move the next input symbol onto the stack.
- **Reduce** — the handle is on top of the stack; replace it by the left side of its production.
- **Accept** — the stack holds $S and the input is $.
- **Error** — call an error-recovery routine.

Handle: a substring that matches the right side of a production, and whose reduction is one step of the reverse rightmost derivation. Formally, if $S \Rightarrow^*_{rm} \alpha A w \Rightarrow_{rm} \alpha\beta w$, then $A \to \beta$ at the position after α is a handle of $\alpha\beta w$. **Handle pruning** means repeatedly finding the handle and replacing it with its left side, until only the start symbol is left. This produces the rightmost derivation in reverse.

Example ($E \to E+E \mid E*E \mid id$, input $id_1 * id_2$): the handles are id_1 ($E \to id$), then id_2 ($E \to id$), then $E*E$ ($E \to E*E$), giving E.

Augmented grammar: for a grammar G with start symbol S, G' is G plus a new start symbol S' and the production **S' → S**. It tells the parser exactly when to stop: the parser accepts only when it is about to reduce by S' → S. This avoids confusion when S also appears on the right side of other productions.

Conflicts: *shift/reduce* (the parser cannot decide whether to shift or reduce) and *reduce/reduce* (it cannot decide which production to use). LR parsers avoid them by using states and lookahead.

4.2 Trace the handle-pruning process while parsing the input "abbcdc" with a shift-reduce parser for S → aABc, A → Abc | b, B → d. List the handles found in each step and the final reduction sequence. **6 M**

Apr-25 Q7c

#	Stack	Input	Action	Handle
1	$	abbcdc$	shift a	
2	$a	bbcdc$	shift b	
3	$ab	bcdc$	reduce A → b	**b**
4	$aA	bcdc$	shift b	
5	$aAb	cdc$	shift c	
6	$aAbc	dc$	reduce A → Abc	**Abc**
7	$aA	dc$	shift d	
8	$aAd	c$	reduce B → d	**d**
9	$aAB	c$	shift c	
10	$aABc	$	reduce S → aABc	**aABc**
11	$S	$	**accept**	

At step 5, b on top of the stack is *not* a handle. Reducing it to A would give aAA, which cannot lead back to S. The correct handle is Abc.

Handles in order: b, Abc, d, aABc
Reduction sequence: abbcdc ⇐ aAbcdc ⇐ aAdc ⇐ aABc ⇐ S
i.e. rightmost derivation: S ⇒ aABc ⇒ aAdc ⇒ aAbcdc ⇒ abbcdc

4.3 Perform shift-reduce parsing for the input string (a,(a,a)) using S → (L) | a, L → L,S | S.

5 M

Aug-24 Q5c

Stack	Input	Action
$	(a,(a,a))$	shift (
$(	a,(a,a))$	shift a
$(a	,(a,a))$	reduce S → a
$(S	,(a,a))$	reduce L → S
$(L	,(a,a))$	shift ,
$(L,	(a,a))$	shift (
$(L,(	a,a))$	shift a
$(L,(a	,a))$	reduce S → a
$(L,(S	,a))$	reduce L → S
$(L,(L	,a))$	shift ,
$(L,(L,	a))$	shift a
$(L,(L,a	))$	reduce S → a
$(L,(L,S	))$	reduce L → L,S
$(L,(L	))$	shift)
$(L,(L)	)$	reduce S → (L)
$(L,S	)$	reduce L → L,S
$(L	)$	shift)
$(L)	$	reduce S → (L)
$S	$	**accept**

4.4 Differentiate between shift-reduce and operator-precedence parsers. 5 M

Aug-24 Q6a

Shift-reduce (LR family)	Operator-precedence parser
General bottom-up technique; works for a large class of CFGs (LR grammars)	A special shift-reduce parser that works only for **operator grammars** (no ε-productions, no two adjacent non-terminals)
Decisions come from parser states plus a lookahead (ACTION/GOTO tables)	Decisions come from precedence relations <·, ≐, ·> between terminals
Finds handles exactly using the DFA of viable prefixes	Handle = the string between <· and ·>; non-terminals are ignored when comparing
Detects errors as soon as possible; precise	Error detection is weaker; may accept some invalid strings
Hard to build by hand; generated by tools (Yacc)	Easy to build by hand for expressions
Can handle unary and binary uses of the same operator, using grammar states	Cannot easily handle an operator with two precedences (e.g. unary vs binary minus)

4.5 Construct the LR(1) / CLR(1) parsing table for S → AA, A → aA | b. (Also: S → CC, C → aC | d.) 7–10 M

Dec-25 Q6b, Q8b Apr-25 Q8b Aug-24 Q8a

Augmented grammar: (0) S' → S (1) S → AA (2) A → aA (3) A → b

An LR(1) item is [A → α•β, a], where a is the **lookahead**. **CLOSURE:** for [A → α•Bβ, a], add [B → •γ, b] for every b ∈ FIRST(βa). **GOTO(I, X)** = CLOSURE of the items with the dot moved past X.

I0 = CLOSURE([S' → •S, $]). For S → •AA, the lookahead of A is FIRST(A$) = {a, b}.

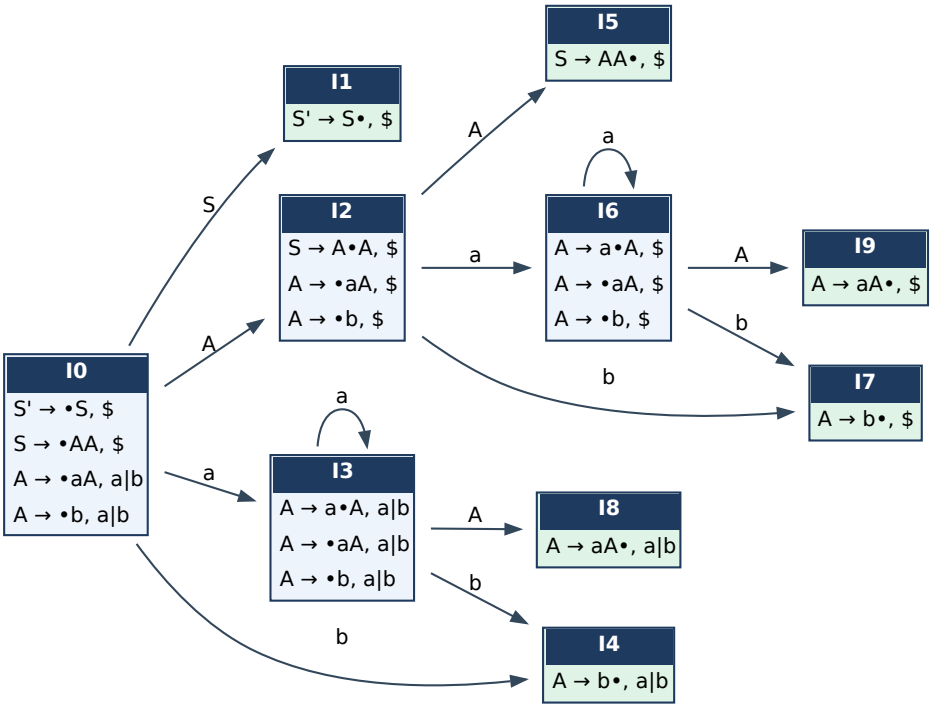


Fig. Canonical collection of LR(1) items (green = states containing a completed item)

CLR(1) parsing table

Rules: if [A → α•aβ, b] ∈ I_i and GOTO(I_i, a) = I_j, set ACTION[i, a] = shift j. If [A → α•, a] ∈ I_i, set ACTION[i, a] = reduce A → α (only on its lookahead a). If [S' → S•, $] ∈ I_i, set ACTION[i, $] = accept.

State	ACTION			GOTO	
	a	b	$	S	A
0	s3	s4		1	2
1			**acc**		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

There are no multiply-defined entries, so the grammar is **CLR(1) / LR(1)**. (LR(1) and CLR(1) are the same parser.)

Parsing "aabb" (to check the table)

Stack	Input	Action
0	aabb$	s3
0 a 3	abb$	s3
0 a 3 a 3	bb$	s4
0 a 3 a 3 b 4	b$	r3 (A→b), goto(3,A)=8
0 a 3 a 3 A 8	b$	r2 (A→aA), goto(3,A)=8
0 a 3 A 8	b$	r2, goto(0,A)=2
0 A 2	b$	s7
0 A 2 b 7	$	r3, goto(2,A)=5
0 A 2 A 5	$	r1 (S→AA), goto(0,S)=1
0 S 1	$	**accept**

Aug-24 version (S → CC, C → aC | d): this is the same grammar with A renamed C and b renamed d. Use the same item sets and the same table, with column **b → d** and GOTO column **A → C**. Productions: (1) S → CC, (2) C → aC, (3) C → d. For example, I4 = [C → d•, a|d] gives r3 on a and d, and I7 = [C → d•, $] gives r3 on $.

4.6 Construct the LALR(1) parsing table for S → AA, A → aA, A → b. 8 M

Dec-25 Q7a Apr-25 Q7a

Method: build the canonical LR(1) collection (see 4.5). Then **merge states that have the same core** (the same LR(0) items, differing only in lookaheads), taking the union of their lookaheads.

Same core	Merged state	Items after merging
I3 and I6	**I36**	A → a•A, a|b|$; A → •aA, a|b|$; A → •b, a|b|$
I4 and I7	**I47**	A → b•, a|b|$
I8 and I9	**I89**	A → aA•, a|b|$

Unchanged: I0, I1, I2, I5. That gives 7 states in total (the same number as SLR/LR(0)).

State	ACTION			GOTO	
	a	b	$	S	A
0	s36	s47		1	2
1			**acc**		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

No conflicts appear after merging (merging can only create reduce/reduce conflicts, and here there are none). So the grammar is **LALR(1)**. The LALR parser detects errors a little later than CLR, but it has far fewer states.

4.7 Construct the LR(0) parsing table for E → E*B | E+B | B, B → 0 | 1. 8 M

Apr-25 Q8c

Augmented grammar: (0) S → E (1) E → E*B (2) E → E+B (3) E → B (4) B → 0 (5) B → 1

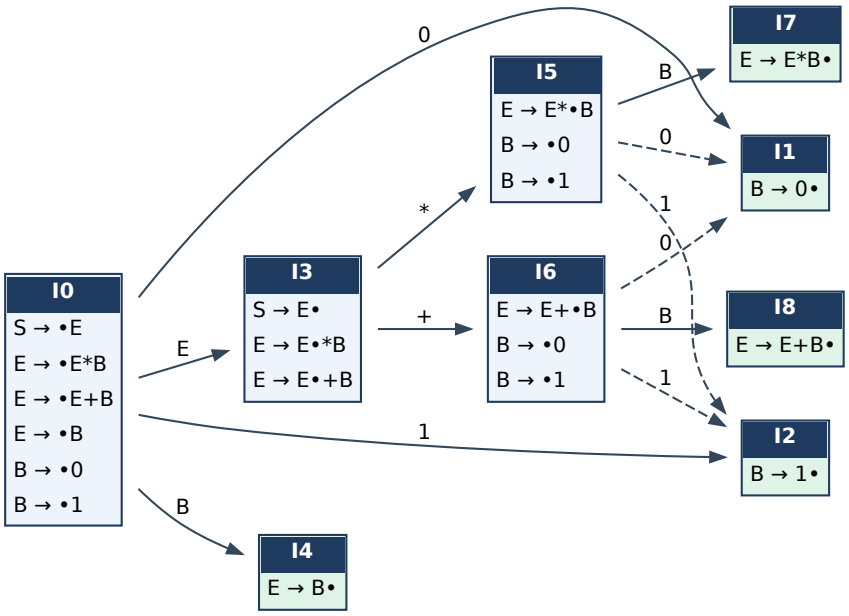


Fig. Canonical collection of LR(0) items and the GOTO graph

LR(0) rule: if a state contains a completed item A → α•, put the reduce action in **every** terminal column. There is no lookahead.

State	ACTION					GOTO	
	*	+	0	1	$	E	B
0			s1	s2		3	4
1	r4	r4	r4	r4	r4		
2	r5	r5	r5	r5	r5		
3	s5	s6			**acc**		
4	r3	r3	r3	r3	r3		
5			s1	s2			7
6			s1	s2			8
7	r1	r1	r1	r1	r1		
8	r2	r2	r2	r2	r2		

No state has both a shift and a reduce, or two reduces, so the grammar is **LR(0)**. Example: "1+1" → s2, r5 (B→1), goto 4, r3 (E→B), goto 3, s6, s2, r5, goto 8, r2 (E→E+B), goto 3, accept.

4.8 Differences between SLR, CLR and LALR parsers. 4 M

Aug-24 Q8c

Feature	SLR(1)	CLR(1) / LR(1)	LALR(1)
Items used	LR(0) items	LR(1) items (core + lookahead)	LR(1) items with the same core merged
Reduce placed on	FOLLOW(A)	the exact lookahead of the item	the union of the merged lookaheads
Number of states	Small	Largest	Same as SLR
Power	Least	Most powerful	In between (SLR ⊂ LALR ⊂ CLR)
Conflicts	Most S/R conflicts	Fewest	May get R/R conflicts from merging
Error detection	Delayed	Immediate	May do a few extra reductions first
Used by	Teaching, simple tools	Rarely (table too large)	Yacc, Bison (practical choice)

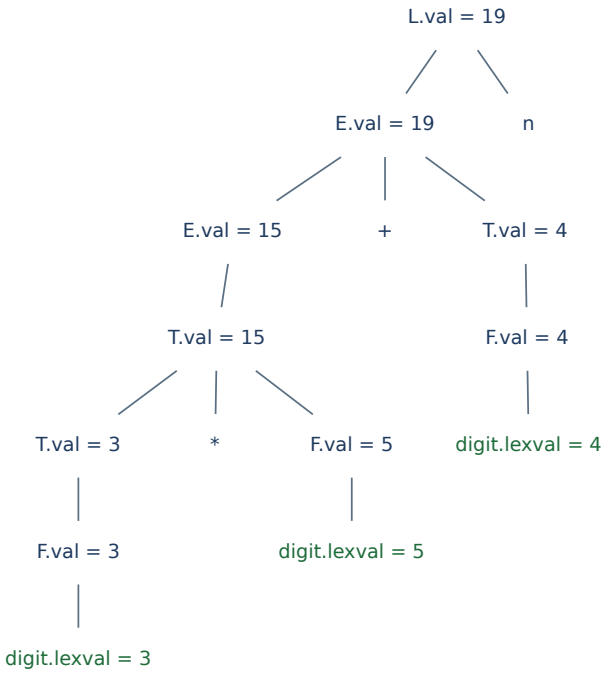
4.9 Explain Syntax-Directed Definitions (SDD) with an example. Differentiate between synthesized and inherited attributes. / Define syntax-directed translation with an example. 4–6 M

Dec-25 Q7b Apr-25 Q8a Aug-24 Q7b

A **Syntax-Directed Definition (SDD)** is a context-free grammar together with **attributes** attached to grammar symbols and **semantic rules** attached to productions. A rule computes the value of an attribute, e.g. E.val = E_1.val + T.val. A **Syntax-Directed Translation (SDT) scheme** instead embeds program fragments called *semantic actions* inside the production bodies, e.g. E → E_1 + T { print('+') }. An SDT also fixes the order in which the actions run.

Example — SDD of a desk calculator (S-attributed)

Production	Semantic rule
L → E n	L.val = E.val
E → E₁ + T	E.val = E₁.val + T.val
E → T	E.val = T.val
T → T₁ * F	T.val = T₁.val × F.val
T → F	T.val = F.val
F → (E)	F.val = E.val
F → digit	F.val = digit.lexval

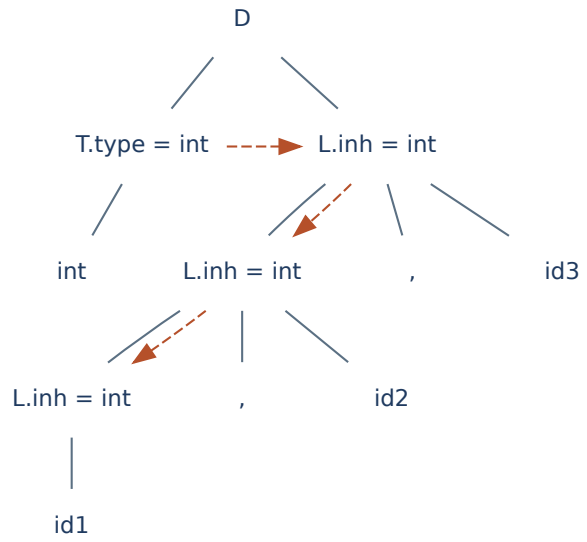


Annotated parse tree for 3 * 5 + 4 n

Synthesized vs inherited attributes

Synthesized attribute	Inherited attribute
Value at node N is computed from the attributes of N's **children** (and N itself)	Value at node N is computed from N's **parent**, its **siblings** and N itself
Information flows **bottom-up**	Information flows **top-down / sideways**
Defined by a rule of the production whose head is N	Defined by a rule of the production whose *body* contains N
Evaluated naturally during bottom-up (LR) parsing	Needs a dependency-graph order, e.g. top-down
An SDD with only these is **S-attributed**	Allowed in **L-attributed** SDDs (from the left only)
e.g. E.val, T.val above	e.g. L.inh in D → T L, rule L.inh = T.type

Production	Rule (type declaration)
D → T L	L.inh = T.type
T → int	T.type = integer
T → float	T.type = float
L → L₁ , id	L₁.inh = L.inh; addType(id.entry, L.inh)
L → id	addType(id.entry, L.inh)



int id1, id2, id3 — the type is passed down (inherited)

4.10 What are the applications of SDD? 6 M

Dec-25 Q8c

1. **Construction of syntax trees:** SDDs with rules such as E.node = new Node('+', E₁.node, T.node) build abstract syntax trees during parsing.
2. **Type checking:** attributes carry type information (E.type) and rules check compatibility, e.g. if E₁.type = int and E₂.type = int then E.type = int, else type_error.
3. **Intermediate code generation:** attributes such as E.addr and E.code generate three-address code, e.g. E.code = E₁.code ‖ E₂.code ‖ gen(E.addr '=' E₁.addr '+' E₂.addr).
4. **Evaluating expressions:** desk calculators (the val attribute), constant folding.
5. **Symbol-table management and declarations:** inherited attributes pass the type to a list of identifiers (D → T L), and the offset or width of variables is computed for storage layout.
6. **Translating control flow:** the attributes B.true, B.false and S.next generate jumps for if, while and Boolean expressions (backpatching).
7. **Infix to postfix translation:** e.g. E → E₁ + T { print('+') }.
8. **Other uses:** pretty printers, document formatting, and generating code for array references.

4.11 Write a note on Yacc. / Explain the working of the Yacc parser generator in detail. 6–8 M

Dec-25 Q7c Apr-25 Q7b Aug-24 Q7a

Yacc ("Yet Another Compiler-Compiler", S.C. Johnson; GNU version Bison) is an **LALR(1) parser generator**. It reads a grammar specification with semantic actions and produces a C program, `y.tab.c`, containing the function `yyparse()` and the LALR parsing tables.

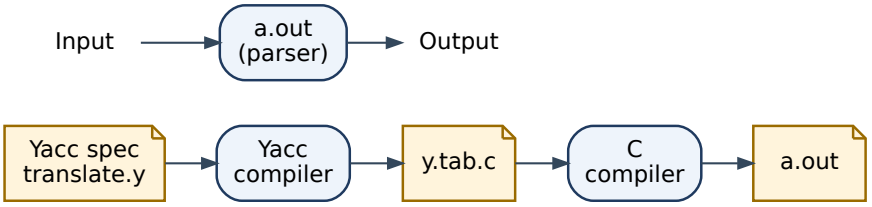


Fig. Creating an input/output translator with Yacc

Structure of a Yacc program

```
declarations
%%
translation rules
%%
supporting C routines
```

- **Declarations:** C code in `%{ … %}`; token declarations `%token DIGIT`; precedence and associativity (`%left '+' '-'`, `%right UMINUS`); `%start`.
- **Rules:** `head : body₁ {action} | body₂ {action} ;` where `$$` is the value of the head and `$1, $2, …` are the values of the body symbols.
- **Routines:** `main()`, `yylex()` (often generated by Lex), `yyerror()`.

```
%{
#include <ctype.h>
%}
%token DIGIT
%left '+'
%left '*'
%%
line : expr '\n'        { printf("%d\n", $1); } ;
expr : expr '+' expr    { $$ = $1 + $3; }
     | expr '*' expr    { $$ = $1 * $3; }
     | '(' expr ')'     { $$ = $2; }
     | DIGIT ;
%%
yylex() { int c = getchar();
          if (isdigit(c)) { yylval = c - '0'; return DIGIT; }
          return c; }
```

Working

1. Yacc builds the LALR(1) item sets and the ACTION/GOTO tables from the rules.
2. `yyparse()` calls `yylex()` for tokens. Lex returns the token code, and its value goes in `yylval`.
3. The parser shifts tokens onto a state stack and a parallel value stack. On a reduce, it runs the rule's action, computing `$$` from `$1…$n`.
4. **Conflict resolution:** for a shift/reduce conflict, the default is to shift; for a reduce/reduce conflict, the earlier rule wins. The `%left/%right` declarations resolve conflicts in ambiguous grammars. `yacc -v` writes the conflicts to y.output.
5. **Error recovery:** the special `error` token, e.g. `stmt : error ';'`, together with `yyerror()`.

Commands: `lex calc.l` → lex.yy.c; `yacc -d calc.y` → y.tab.c, y.tab.h; `cc y.tab.c lex.yy.c -ly -ll` → a.out.

4.12 Discuss type checking with suitable examples. 8 M

Aug-24 Q7c

Type checking is the part of semantic analysis that makes sure the type of every construct matches what its context expects, according to the language's **type system** (a set of rules assigning type expressions to program parts). Example: an arithmetic operator applied to an integer and a function is an error.

Static vs dynamic checking

- **Static:** done at compile time (C, Java). It catches errors early and adds no runtime cost.
- **Dynamic:** done at run time (Python, array-bound checks). It is more flexible but slower.
- A **strongly typed** language guarantees that accepted programs run without type errors.

Type expressions

Basic types: int, float, char, boolean, void, type_error. Constructors: array(I, T), pointer(T), product $T_1 \times T_2$, function $T_1 \to T_2$, record(...). For example, `int a[10]` has type array(0..9, int), and `float f(int, char)` has type int × char → float.

Type-checking rules (as an SDD)

Production	Semantic rule
E → num	E.type = integer
E → id	E.type = lookup(id.entry)
$E \to E_1 + E_2$	if E_1.type = int and E_2.type = int then int; else if either is float then float (coerce); else type_error
$E \to E_1 [E_2]$	if E_2.type = int and E_1.type = array(s, t) then t else type_error
$E \to *E_1$	if E_1.type = pointer(t) then t else type_error
$E \to E_1 (E_2)$	if E_1.type = s → t and E_2.type = s then t else type_error
S → id = E	if id.type = E.type then void else type_error
S → if E then S_1 / while E do S_1	if E.type = boolean then S_1.type else type_error

Key concepts

- **Type conversion / coercion:** in x = i * 2.5 with i an int, the compiler inserts inttofloat(i). This is an implicit (widening) conversion. An explicit conversion is a cast: (int) f.
- **Type equivalence:** *structural* (same structure) vs *name* equivalence (same type name).
- **Overloading:** '+' means integer addition, float addition or string concatenation depending on the operand types. It is resolved from the types.
- **Polymorphism and type inference:** e.g. a generic length: list(α) → int, where the type is inferred using unification.

Example of errors caught: `int x; char *p; x = p + 1.5;` → error: pointer + float is invalid; `if (x) …` where x is a struct → error: the condition is not a boolean or scalar.

UNIT V

Intermediate Code, Run-Time Storage & Optimization

Syntax trees & DAGs · three-address code · quadruples/triples · basic blocks & flow graphs · storage organization · optimization

5.1 Explain the different types of intermediate code with an example. / Briefly explain the variants of syntax trees.

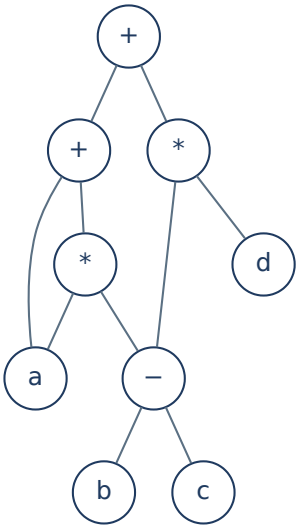
7–8 M

Aug-24 Q10a Apr-25 Q9a

The **intermediate representation (IR)** sits between the front end and the back end. It makes retargeting easy and allows machine-independent optimization. Common forms:

1. **Syntax tree (abstract syntax tree):** each interior node is an operator and its children are its operands. It is a condensed parse tree with no chain productions or punctuation.
2. **Directed Acyclic Graph (DAG):** like a syntax tree, but a common subexpression is represented by *one* shared node, which can have several parents.
3. **Postfix notation:** the operator comes after its operands, e.g. a + b * c → `a b c * +`. It can be evaluated with a stack.
4. **Three-address code (TAC):** a sequence of instructions x = y op z, with at most one operator on the right side. It is stored as **quadruples**, **triples** or **indirect triples**.
5. **Static Single Assignment (SSA):** TAC in which each name is assigned exactly once, using φ-functions where control flow joins.

Variants of syntax trees, for a + a * (b − c) + (b − c) * d



DAG: a and (b − c) are shared

Syntax tree vs DAG: the syntax tree would build two separate nodes for (b − c) and two leaves for a. The DAG detects that they are the same and reuses the node. So the DAG tells the code generator that b − c needs to be computed only **once**.

Building the DAG: before creating a node (op, l, r), the constructor *Node* checks whether an identical node already exists. If it does, the existing node is returned.

Value-number method

The nodes are stored in an array of records. Each node is referred to by its index, called its **value number**. A record is <op, left, right>. To find an existing node quickly, a hash table is keyed on the signature (op, l, r). For i = i + 10:

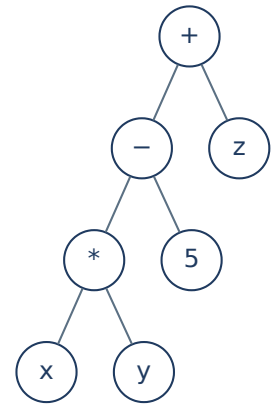
#	op	left	right
1	id	→ entry for i	
2	num	10	
3	+	1	2
4	=	1	3

TAC for the same expression: t1 = b − c; t2 = a * t1; t3 = a + t2; t4 = t1 * d; t5 = t3 + t4.
Postfix: a a b c − * + b c − d * +

5.2 Construct the syntax tree for the expression x * y − 5 + z. — 8 M

Dec-25 Q9b

* has higher precedence than − and +, and − and + are left-associative with equal precedence. So the expression groups as **((x * y) − 5) + z**.



Syntax tree for x*y−5+z

Construction steps (bottom-up, using Leaf and Node constructors):

```
p1 = new Leaf(id, entry-x)
p2 = new Leaf(id, entry-y)
p3 = new Node('*', p1, p2)
p4 = new Leaf(num, 5)
p5 = new Node('-', p3, p4)
p6 = new Leaf(id, entry-z)
p7 = new Node('+', p5, p6)   ← root
```

The SDD that builds it: E → E₁ + T { E.node = new Node('+', E₁.node, T.node) }; E → E₁ − T { E.node = new Node('−', E₁.node, T.node) }; T → T₁ * F { T.node = new Node('*', T₁.node, F.node) }; F → id { F.node = new Leaf(id, id.entry) }; F → num { F.node = new Leaf(num, num.val) }.

Corresponding TAC: t1 = x * y; t2 = t1 − 5; t3 = t2 + z. **Postfix:** x y * 5 − z +

5.3 Construct the quadruples, triples and indirect triples of the expression a * −(b + c). — 8 M

Dec-25 Q10b

Three-address code: t1 = b + c; t2 = minus t1; t3 = a * t2 (and a = t3 if the result is assigned)

Quadruples (op, arg1, arg2, result)

#	op	arg1	arg2	result
(0)	+	b	c	t1
(1)	minus	t1		t2
(2)	*	a	t2	t3

Triples (op, arg1, arg2)

#	op	arg1	arg2
(0)	+	b	c
(1)	minus	(0)	
(2)	*	a	(1)

Indirect triples

Statement	Pointer
(0)	(14)
(1)	(15)
(2)	(16)

#	op	arg1	arg2
(14)	+	b	c
(15)	minus	(14)	
(16)	*	a	(15)

Quadruples	Triples	Indirect triples
Temporaries are named explicitly, which needs symbol-table entries	No temporaries: a result is referred to by its position (i)	A separate list of pointers to the triples
Easy to move code during optimization	Moving a statement changes every reference to it	Reorder the pointer list only, so code is easy to move
Uses more space	Uses less space	Space in between

5.4 Write the three-address code, quadruples, triples and indirect triples for: (1) a + b + (c + d) + d + e + a (2) a = (−c * b) + (−c * d) — 10 M

Apr-25 Q10b

(1) a + b + (c + d) + d + e + a (+ is left-associative)

```
t1 = a + b
t2 = c + d
t3 = t1 + t2
t4 = t3 + d
t5 = t4 + e
t6 = t5 + a
```

#	op	arg1	arg2	result
(0)	+	a	b	t1
(1)	+	c	d	t2
(2)	+	t1	t2	t3
(3)	+	t3	d	t4
(4)	+	t4	e	t5
(5)	+	t5	a	t6

Triples

#	op	arg1	arg2
(0)	+	a	b
(1)	+	c	d
(2)	+	(0)	(1)
(3)	+	(2)	d
(4)	+	(3)	e
(5)	+	(4)	a

Indirect triples

Stmt	Ptr
(0)	(20)
(1)	(21)
(2)	(22)
(3)	(23)
(4)	(24)
(5)	(25)

Triples (20)–(25) are the same as (0)–(5) above, with references renumbered: (22) = + (20) (21), and so on.

(2) a = (−c * b) + (−c * d)

```
t1 = minus c
t2 = t1 * b
t3 = minus c
t4 = t3 * d
t5 = t2 + t4
a  = t5
```

#	op	arg1	arg2	result
(0)	minus	c		t1
(1)	*	t1	b	t2
(2)	minus	c		t3
(3)	*	t3	d	t4
(4)	+	t2	t4	t5
(5)	=	t5		a

Triples

#	op	arg1	arg2
(0)	minus	c	
(1)	*	(0)	b
(2)	minus	c	
(3)	*	(2)	d
(4)	+	(1)	(3)
(5)	=	a	(4)

Indirect triples

Stmt	Ptr
(0)	(35)
(1)	(36)
(2)	(37)
(3)	(38)
(4)	(39)
(5)	(40)

Optimization note: minus c is a common subexpression, so t3 can be replaced by t1.

5.5 (i) Translate A[i] := b and a := B[i] into triple representation. (ii) Generate the three-address code, compute the basic blocks and draw the flow graph for: prod = 0; i = 1; do { prod = prod + a[i] * b[i]; i = i + 1; } while (i ≤ 10); — 8 M

Aug-24 Q9c

(i) An indexed assignment needs **two** triples, because a triple has only two arguments:

A[i] := b			
#	op	arg1	arg2
(0)	[ ]=	A	i
(1)	assign	(0)	b

a := B[i]			
#	op	arg1	arg2
(0)	=[ ]	B	i
(1)	assign	a	(0)

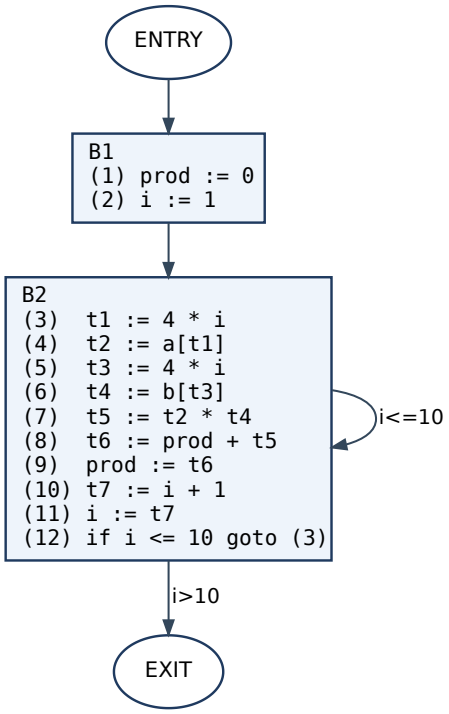
(ii) TAC (each array element is 4 bytes wide):

```
(1)  prod := 0
(2)  i := 1
(3)  t1 := 4 * i
(4)  t2 := a[t1]
(5)  t3 := 4 * i
(6)  t4 := b[t3]
(7)  t5 := t2 * t4
(8)  t6 := prod + t5
(9)  prod := t6
(10) t7 := i + 1
(11) i := t7
(12) if i <= 10 goto (3)
```

Leaders (rules: the first statement; the target of a jump; the statement after a jump):

- (1) — first statement
- (3) — target of goto in (12)

Basic blocks: B1 = (1)–(2), B2 = (3)–(12)



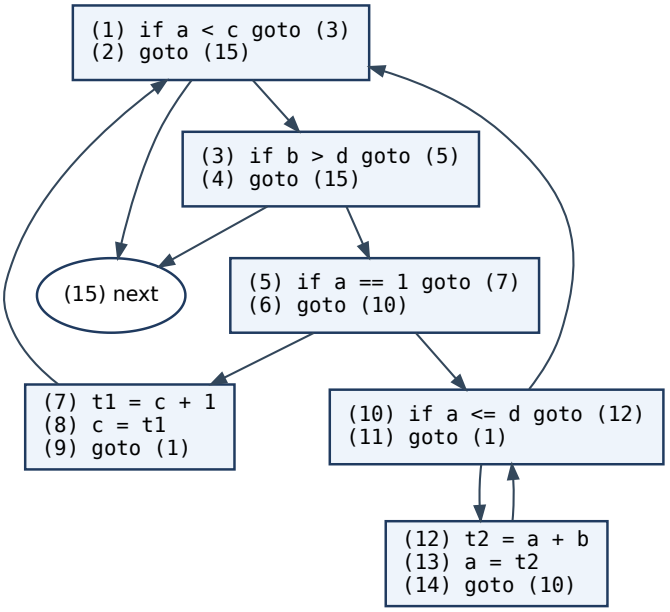
5.6 Generate three-address code for: while (a < c and b > d) do if a = 1 then c = c + 1 else while a <= d do a = a + b

8 M

Apr-25 Q9b

The Boolean **and** is translated with short-circuit (jumping) code. If *a < c* is false, the loop ends at once without testing *b > d*.

```
(1)  if a < c goto (3)
(2)  goto (15)
(3)  if b > d goto (5)
(4)  goto (15)
(5)  if a == 1 goto (7)
(6)  goto (10)
(7)  t1 = c + 1
(8)  c = t1
(9)  goto (1)
(10) if a <= d goto (12)
(11) goto (1)
(12) t2 = a + b
(13) a = t2
(14) goto (10)
(15) next ...
```



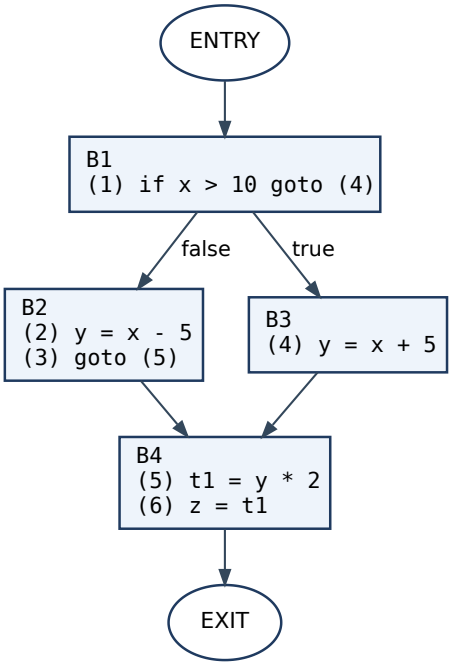
(1)–(4) test the outer condition; (5)–(6) the if; (7)–(9) the then-part, which jumps back to the outer loop test; (10)–(14) the inner while loop, whose exit (11) goes back to the outer loop test.

5.7 Construct a basic-block flow graph for: if (x > 10) y = x + 5; else y = x − 5; z = y * 2; and identify the number of basic blocks. 7 M

Dec-25 Q9a

Three-address code

```
(1) if x > 10 goto (4)
(2) y = x - 5
(3) goto (5)
(4) y = x + 5
(5) t1 = y * 2
(6) z = t1
```



Finding leaders

- (1) is the first statement
- (2) follows a conditional jump
- (4) is the target of the jump in (1)
- (5) is the target of the goto in (3)

Basic blocks: B1 = {1}, B2 = {2, 3}, B3 = {4}, B4 = {5, 6}

Number of basic blocks = 4. Edges: B1→B2 (condition false), B1→B3 (condition true), B2→B4, B3→B4.

5.8 Define a flow graph. Differentiate between a basic block and a flow graph. Explain the structure-preserving transformations for optimizing a basic block. 4–5 M

Dec-25 Q10c Aug-24 Q9a

Basic block: a maximal sequence of consecutive three-address statements that control enters only at the first statement and leaves only at the last, with no halt or branch in between. **Flow graph:** a directed graph whose nodes are the basic blocks. There is an edge B1 → B2 if B2 can immediately follow B1 during execution: either B1 ends in a jump to B2's leader, or B2 comes right after B1 and B1 does not end in an unconditional jump.

Basic block	Flow graph
A straight-line sequence of instructions	A graph made of basic blocks connected by control-flow edges
Single entry, single exit; no internal branches	Contains branches, loops and join points
Found using the *leader* algorithm	Built by adding edges between the blocks
Unit of **local** optimization (DAG, CSE)	Used for **global** optimization (data-flow and loop analysis)

Structure-preserving transformations (local, within a block)

1. **Common subexpression elimination:** a = b + c; b = a − d; c = b + c; d = a − d ⟹ the second a − d is replaced: d = b.
2. **Dead-code elimination:** remove x = y + z if x is never used afterwards (x is dead).
3. **Renaming temporary variables:** t = b + c can become u = b + c (with all uses of t changed). This gives a normal-form block in which each temporary is defined once.
4. **Interchange of statements:** t1 = b + c; t2 = x + y can be swapped if neither uses the other's result. This helps scheduling.

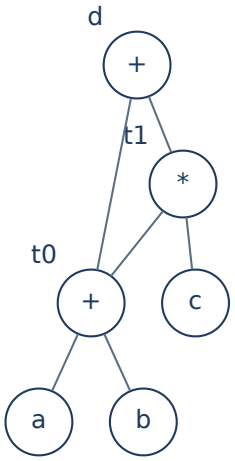
5.9 What is a Directed Acyclic Graph (DAG)? Draw the DAG for t0 = a + b; t1 = t0 * c; d = t0 + t1. — 7 M

Aug-24 Q9b

A **DAG** for a basic block is a directed graph with no cycles in which:

- **leaves** are labelled by identifiers or constants (the initial values of names, e.g. a_0);
- **interior nodes** are labelled by operators;
- each node can also carry a list of **identifiers** (attached names) that hold its value.

Uses: finding common subexpressions automatically, finding which names are used inside the block but defined outside, finding which statements compute values used outside the block, and generating optimized code.



DAG: node (+ a b) is shared by t1 and d

Construction:

1. t0 = a + b → create leaves a, b and node + (label t0).
2. t1 = t0 * c → leaf c; node * (t0, c), label t1.
3. d = t0 + t1 → node + (t0-node, t1-node), label d.

The t0 node has two parents. This shows that a + b is computed once and reused.

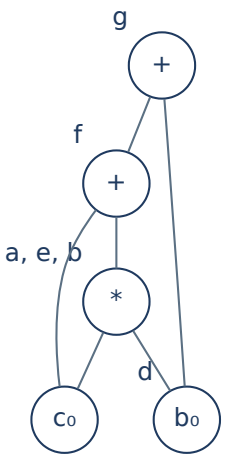
5.10 Construct the DAG for the block: a := b*c; d := b; e := d*c; b := e; f := b + c; g := f + d

10 M

Apr-25 Q10a

Algorithm: for each statement x := y op z: find or create nodes for y and z (by their current values), look for an existing node (op, y, z) and create one only if none exists, then attach the label x to it (removing x from any node it labelled before). For x := y, just attach x to y's node.

Statement	Action on the DAG
a := b * c	Create leaves b_0, c_0 and node n1 = (* b_0 c_0); label **a**
d := b	Copy: attach label **d** to leaf b_0
e := d * c	d is the node b_0, so this is (* b_0 c_0), which is n1 — **common subexpression**. Attach **e** to n1
b := e	Attach **b** to n1. b now means n1; leaf b_0 keeps the old value
f := b + c	Node n2 = (+ n1 c_0); label **f**
g := f + d	Node n3 = (+ n2 b_0); label **g**



Final DAG

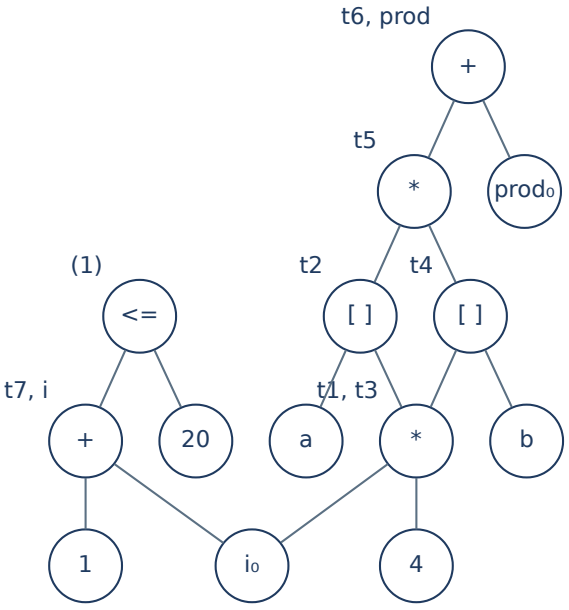
Optimized code from the DAG (assuming all names are live on exit):

```
a := b * c
d := b
f := a + c
g := f + d
e := a
b := a
```

The second multiplication `d * c` is removed (common subexpression). If e is not live after the block, `e := a` is also removed (dead code). Note that `d := b` must come before `b := a`.

5.11 Construct the DAG for the basic block: t1 := 4*i; t2 := a[t1]; t3 := 4*i; t4 := b[t3]; t5 := t2*t4; t6 := prod + t5; prod := t6; t7 := i + 1; i := t7; if i <= 20 goto (1) — 6 M

Aug-24 Q10b



DAG for the basic block

- t1 := 4*i creates node (* 4 i_0), label t1.
- t3 := 4*i finds the **same** node, so it gets labels t1, t3.
- t2 := a[t1] and t4 := b[t3] become two [ ] nodes sharing the (* 4 i_0) child.
- t5 = (* t2 t4); t6 = (+ $prod_0$ t5); prod := t6 attaches prod to it.
- t7 = (+ i_0 1); i := t7 attaches i.
- (<= t7-node 20) is the conditional jump back to (1).

Optimized block:

```
t1 := 4 * i
t2 := a[t1]
t4 := b[t1]
t5 := t2 * t4
prod := prod + t5
i := i + 1
if i <= 20 goto (1)
```

5.12 Explain the principal sources of optimization. — 5 M

Dec-25 Q9c

A transformation must **preserve the meaning** of the program, make it measurably faster or smaller, and be worth the effort. The main function-preserving transformations are:

1. **Common subexpression elimination:** if E was computed before and its operands have not changed, reuse the value. `t1 = 4*i; … t3 = 4*i` ⟹ `t3 = t1`.
2. **Copy propagation:** after x = y, use y in place of x. This often makes x dead.
3. **Dead-code elimination:** remove statements whose results are never used, or code that can never be reached (`if (debug)` with debug = false).
4. **Constant folding and propagation:** evaluate constant expressions at compile time: `x = 2 * 3.14` ⟹ `x = 6.28`.
5. **Loop optimizations** (most programs spend most of their time in inner loops):
   - *Code motion:* move loop-invariant computations out of the loop: `while (i <= limit-2)` ⟹ `t = limit-2; while (i <= t)`.
   - *Induction-variable elimination:* when i and t = 4*i change in step, keep only one of them.
   - *Strength reduction:* replace an expensive operation with a cheaper one: `t = 4*i` inside a loop ⟹ `t = t + 4`.

5.13 Discuss copy propagation, dead-code elimination and strength reduction in detail.

4–7 M

Apr-25 Q9c Aug-24 Q10c

Copy propagation

A **copy statement** has the form f = g. Copy propagation replaces later uses of f with g, as long as neither f nor g is reassigned in between. It does not save time by itself, but it creates dead code.

Original	After copy propagation	After dead-code elimination (x unused)

`x = a;`
`y = x * 2;`
`z = x + y;`
	`x = a;`
	`y = a * 2;`
	`z = a + y;`
		`y = a * 2;`
		`z = a + y;`

Dead-code elimination

A variable is **live** at a point if its value may be used later; otherwise it is **dead**. Statements that compute values that are never used, and code that cannot be reached, can be removed safely. This makes the program smaller and faster.

```
debug = false;
if (debug) printf("value=%d", x);   ⇒  the whole if is removed (always false)
t = a + b;  /* t never used */       ⇒  removed
return x;  x = x + 1;                ⇒  x = x + 1 is unreachable, removed
```

Dead code often comes from earlier optimizations (copy propagation, constant folding) or from macros and debug code. It is found using liveness (data-flow) analysis and by checking which blocks are reachable in the flow graph.

Strength reduction

Replace an **expensive** operation with an equivalent **cheaper** one, especially for induction variables inside loops:

Before	After
x * 2	x + x or x << 1
x ** 2	x * x
x / 8 (unsigned)	x >> 3

`for (i = 1; i <= n; i++)`
`  t = 4 * i;  /* multiply each time */`
	`t = 0;`
	`for (i = 1; i <= n; i++)`
	`  t = t + 4;  /* add only */`

Since t = 4·i increases by 4 each time i increases by 1, the multiplication is replaced by an addition. If i is then used only to control the loop, induction-variable elimination can remove i as well.

5.14 Explain storage organization in detail.

8 M

Dec-25 Q10a

At run time the target program executes in its own logical address space. The compiler, OS and target machine together decide how that space is divided. A typical layout is:

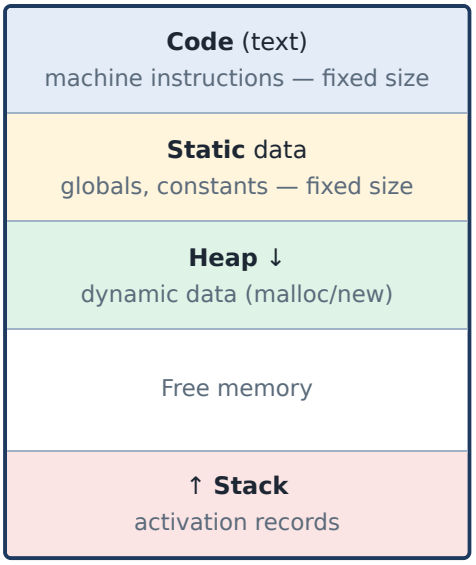


Fig. Run-time memory subdivision

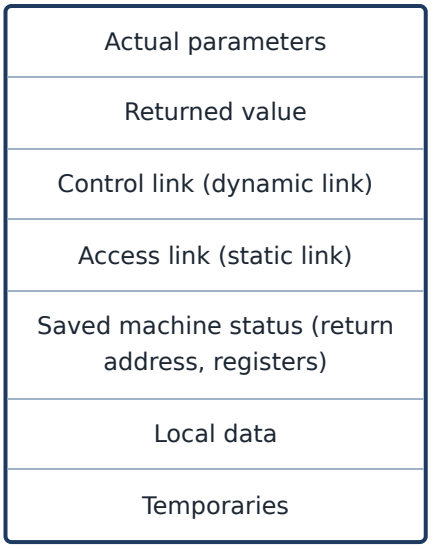


Fig. A general activation record

- **Code area:** the size of the generated code is fixed at compile time, so it is placed in a statically determined area, usually at the low end of memory.
- **Static area:** global variables, static locals and constants. Their size is known at compile time, and their addresses can be compiled into the code.
- **Stack:** holds the **activation records** of procedure calls. A record is pushed when a procedure is called and popped when it returns. Local variables live only as long as the call.
- **Heap:** data whose lifetime is not tied to calls, such as objects created with malloc/new and linked lists. It is freed explicitly (free/delete) or by garbage collection. The stack and the heap grow towards each other.

Storage-allocation strategies

Strategy	How it works	Limitations
Static allocation	Names are bound to storage at compile time; the binding never changes.	No recursion; data structure sizes must be known at compile time; no dynamic data.
Stack allocation	An activation record is pushed on each call and popped on return. Locals are addressed relative to the frame pointer.	Local values are lost when the call ends (no dangling locals).
Heap allocation	Blocks are allocated and freed in any order (first-fit, best-fit; garbage collection).	Slower; fragmentation; memory leaks.

Activation record fields: *actual parameters* passed by the caller; *returned value*; *control link*, which points to the caller's record; *access link*, which points to the enclosing scope's record so non-local data can be reached; *saved machine status*, i.e. the return address and registers; *local data*; and *temporaries* used while evaluating expressions. The **calling sequence** allocates the record and fills these fields. The **return sequence** restores the machine state and deallocates the record.